# Do LLMs Shape the Diffusion of New Software?
# Training Cutoffs and Adoption Dynamics in the Python Ecosystem

Roland Tuboly
Social Data Science MSc, 1st year

Supervisor: Johannes Wachs
Institute of Data Analytics and Information Systems

Corvinus University of Budapest
2026

## Abstract

Large language models recommend software by drawing on parametric knowledge frozen at their training cutoff. I ask whether that cutoff reshapes which libraries developers adopt. The setting is the Python ecosystem around the September 2021 boundary shared by GPT-3.5 and GPT-4, using weekly PyPI download counts for 527,361 packages and GitHub import data for 28,243 matched libraries.

Identification comes from a Regression Discontinuity Design around the documented cutoff, with a nine-week asymmetric donut for libraries whose training inclusion is ambiguous. The main causal estimate is a Difference-in-Discontinuities, which subtracts a seasonal baseline estimated from 2018–2020 placebo cohorts at the same calendar position. The bandwidth is fixed at $h = 13$ weeks, and standard errors are cluster-robust at the year-by-week level.

Before ChatGPT, libraries on either side of the September 2021 cutoff differed by only about 28% in weekly downloads, which is the autumn release premium visible in all three placebo years. This gap starts to widen more and more once ChatGPT becomes available to the public. By early 2026, pre-cutoff libraries average approximately five times more weekly downloads than libraries released just a few weeks later, with no sign of catch-up. The baseline-adjusted Diff-in-RDD gives $p = 0.002$ for the 2021-specific suppression in case of libraries that demonstrated pre-ChatGPT quality. Conditioning on that pre-ChatGPT quality makes the reversal more conservative, not less, because post-cutoff libraries were actually the more successful group before ChatGPT arrived. A Diff-in-RDD on the GitHub-matched subsample finds directionally larger effects for code imports than for downloads, consistent with LLM-steered code generation as the operative channel, though the smaller matched sample is not statistically significant on usual significance levels. No comparable discontinuity appears in any placebo year.

I interpret these results as evidence consistent with a Knowledge Wall, a structural asymmetry in the information landscape of deployed LLMs that may redirect adoption toward libraries already represented in model memory.

It is important to note that this paper has limitations. The running variable is a noisy proxy for true release timing, the mechanism moderation test is underpowered and relies on a post-treatment moderator, and the estimates are local to one cutoff in one ecosystem.



*Keywords* - Regression Discontinuity Design, Large Language Models, Training Data Cutoff, Software Adoption, Python Ecosystem, Knowledge Diffusion.

# Contents


**1 Introduction** **1**

1.1 The Problem . . . 1
1.2 The Mechanism . . . 1
1.3 The Research Gap . . . 2
1.4 Research Setting . . . 2
1.5 Research Questions and Hypotheses . . . 3
1.6 Design Overview . . . 3
1.7 Preview of Findings . . . 3
1.8 Contribution . . . 4
1.9 Paper Structure . . . 5

**2 Literature Review** **6**

2.1 AI, Creativity, and Collective Diversity . . . 6
2.2 Algorithmic Monoculture and Platform Concentration . . . 7
2.3 Technology Diffusion and Innovation Ecology . . . 8
2.4 LLMs in Economic Activity . . . 9
2.5 LLM Knowledge Boundaries . . . 10
2.6 Contribution of this Paper . . . 11

**3 Data** **13**

3.1 Overview . . . 13
3.2 PyPI Download Data . . . 13
3.2.1 Source and Coverage . . . 13
3.2.2 Unit of Observation . . . 13
3.2.3 Release Date Construction . . . 13
3.2.4 Outcome Construction . . . 14
3.2.5 Raw Universe . . . 14
3.3 GitHub Implementation Data . . . 14
3.3.1 Source and Coverage . . . 14
3.3.2 AI Scoring . . . 14
3.3.3 GitHub Outcome Variables . . . 15
3.3.4 Linkage to PyPI . . . 15
3.4 Sample Construction and Attrition . . . 15
3.4.1 Step 1 - Time window restriction . . . 15
3.4.2 Step 2 - Minimum download filter . . . 16
3.4.3 Step 3 - Donut exclusion . . . 16

3.4.4 Missing GitHub Values . . . 16
3.5 Success Tiers . . . 16
3.6 Key Milestones and Calendar Reference Points . . . 17
3.7 Measurement Limitations . . . 18
3.7.1 Release Date Proxy . . . 18
3.7.2 PyPI Download Inflation . . . 18
3.7.3 GitHub Coverage and Selection . . . 18
3.7.4 AI Score Endogeneity . . . 19
3.7.5 Package Identity . . . 19

**4 Empirical Design** **20**
4.1 Overview . . . 20
4.2 Running Variable and Cutoff Date . . . 20
4.2.1 Measurement threats . . . 21
4.3 Treatment Assignment and Estimand . . . 21
4.4 Donut Exclusion . . . 21
4.5 Outcome Variables . . . 22
4.5.1 PyPI downloads . . . 22
4.5.2 GitHub imports . . . 22
4.6 Local Linear RDD Estimator . . . 23
4.7 Identification with Difference-in-Discontinuities . . . 23
4.7.1 Implementation . . . 24
4.7.2 Identifying Assumption . . . 24
4.8 Success Tiers and the Barrier-to-Entry Test . . . 24
4.9 AI Exposure Moderation . . . 25
4.10 Robustness Checks . . . 25
4.10.1 Bandwidth Sensitivity . . . 25
4.10.2 Donut Specification . . . 25
4.10.3 Placebo Cutoffs . . . 26
4.10.4 Density Test . . . 26
4.10.5 Covariate Balance . . . 26
4.11 Permutation Inference . . . 26

**5 Results** **27**
5.1 Sequence of Evidence . . . 27
5.2 Descriptive Patterns . . . 27
5.3 Raw Discontinuity at the 2021 Cutoff . . . 28
5.4 Difference-in-Discontinuities Estimates . . . 29
5.5 Pre-AI Covariate Balance . . . 30
5.6 The Implementation Gap - GitHub vs PyPI . . . 31

5.7 Mechanism Moderation Test . . . . . 34
5.8 Robustness Checks . . . . . 34

**6 Discussion** **36**
6.1 What the Evidence Shows . . . . . 36
6.2 The Knowledge Wall . . . . . 36
6.3 Connection to Doshi and Hauser (2024) . . . . . 37
6.4 Alternative Mechanisms . . . . . 38
6.5 The Systemic Wall Interpretation . . . . . 39
6.6 Scope Conditions and External Validity . . . . . 40
6.7 Limitations . . . . . 40
6.8 Implications . . . . . 41

**7 Conclusion** **42**

**A Declaration of AI Tool Usage** **49**

**B Permutation Inference** **50**

**C Alternative Mechanisms (Extended)** **51**

**D Scope Conditions and Detailed Limitations** **53**

**E Bandwidth and Donut Robustness** **55**

**F Mechanism Moderation Test** **56**

**G Raw Long-Horizon Trajectory** **57**

# List of Tables


3.1 PyPI Download Outcome Variables . . . . . 14
3.2 GitHub Import Outcome Variables . . . . . 15
3.3 Sample Sizes by Cohort Year . . . . . 16
3.4 Success Tier Definitions . . . . . 17
3.5 Key Calendar Reference Points . . . . . 17

5.1 Descriptive Statistics for the 2021 Cohort . . . . . 27
5.2 Main RDD Estimates, 2021 Cohort . . . . . 28
5.3 Diff-in-RDD Estimates, 2021 Excess Jump . . . . . 31
5.4 Pre-AI Covariate Balance, Probability of Reaching the Successful Tier . . . . . 31
5.5 Single-Year RDD Estimates, GitHub Implementation (2021 Only) . . . . . 33
5.6 Diff-in-RDD, GitHub versus PyPI on the Matched Subsample . . . . . 33

F.1 AI Exposure Moderation (Exploratory) . . . . . 56

# List of Figures


1.1 Timeline of the natural experiment. The exogenously defined September 2021 training data cutoff assigns libraries released before the boundary to the treatment group (included in training data) and those released after to the control group (excluded), as-if randomly. The 9-week donut excludes libraries with ambiguous training inclusion. The Knowledge Wall effect activates only after November 2022, when ChatGPT provides a mass interface through which the training boundary becomes consequential for developer behavior. *Source:* Author's own elaboration. . . . . 4

3.1 Density of library releases by week relative to the September 2021 cutoff. The dashed red line marks the cutoff. No systematic bunching is evident at the boundary, consistent with the assumption that release timing is not strategically manipulated around the LLM training cutoff. *Source:* Author's calculations based on PyPI data. . . . . 18

5.1 Binscatter of cumulative PyPI downloads (26 and 52 weeks) . . . . . 29

5.2 Binned scatterplot of $\log(1 + 52\text{-week imports})$ by release week relative to the September 2021 cutoff (GitHub-matched subsample). Each point represents a weekly bin average. The upward trend reflects the secular growth of the Python ecosystem; the discontinuity at the cutoff captures the raw RDD effect, which mixes the Knowledge Wall penalty with the seasonal baseline. *Source:* Author's calculations based on GitHub data. . . . . 30

5.3 Diff-in-RDD visualization for the Successful tier (min. 500 downloads at 26 weeks) - Blue circles: pooled 2018–2020 placebo cohorts. Orange crosses: 2021 cohort. Fitted lines show local linear estimates on each side of the cutoff. The 2021 cohort is shifted downward relative to the historical seasonal norm, identifying the Knowledge Wall penalty. *Source:* Author's calculations based on PyPI data. . . . . 32

5.4 RDD point estimates for PyPI downloads across adoption horizons (Broad tier). Vertical bars are 95% robust confidence intervals. The estimate grows from the 52-week horizon (mostly pre-ChatGPT) through the all-time horizon, consistent with the activation of the Knowledge Wall after November 2022. *Source:* Author's calculations based on PyPI data. . . . . 34

5.5 Normalized diffusion gap between pre- and post-cutoff libraries (PyPI) - The $y$-axis plots the log ratio of mean weekly downloads for the July 2021 (pre-cutoff) cohort relative to the October 2021 (post-cutoff) cohort; values above zero indicate a pre-cutoff advantage. The shaded band shows 95% bootstrap confidence intervals (500 replications, resampled at the library level). Before ChatGPT (November 2022), the ratio fluctuates around a modest baseline of roughly 28%. After ChatGPT, the gap widens steadily and reaches approximately 5:1 by early 2026, with no evidence of closing. *Source:* Author's calculations based on PyPI data. . . . . 35

6.1 Diff-in-RDD excess jump (relative to 2018–2020 placebo average) by success tier. All estimates on the $\log(1+Y)$ scale. The suppression is concentrated in the Broad and Successful tiers; the Superstar tier shows no significant penalty, consistent with high-visibility libraries maintaining adoption through channels independent of LLM recommendation. *Source:* Author's calculations based on PyPI data. . . . . 38

6.2 Cumulative average PyPI downloads for the July 2021 (pre-cutoff) and October 2021 (post-cutoff) cohorts. The vertical red line marks ChatGPT release (November 2022). The gap between cohorts widens monotonically after November 2022, with no catch-up through January 2026. By early 2026, pre-cutoff libraries have accumulated roughly five times the weekly download volume of post-cutoff libraries. *Source:* Author's calculations based on PyPI data. 39

B.1 RDD coefficient distribution across 157 placebo cutoff dates . . . . . 50

E.1 Bandwidth sensitivity of the Diff-in-RDD excess jump (Successful tier, min. 500 downloads at 26 weeks). Points are WLS point estimates; vertical bars are 95% confidence intervals from year-by-week cluster-robust standard errors. The estimate is negative and stable across $h \geq 13$. *Source:* Author's calculations based on PyPI data. . . . . 55

G.1 Weekly PyPI downloads, July vs October 2021 cohorts (log scale) . . . . . 57

# Chapter 1 – Introduction[1]

## 1.1 The Problem

Large language models (LLMs) have become a standard channel through which software developers discover libraries, generate code, and debug. GitHub Copilot, ChatGPT, and the subsequent models are now integrated in development workflows, from novice programmers to professional engineers at large technology firms. By 2024, more than half of professional developers said they regularly use AI tools to help them write code (Stack Overflow, 2024), and the share of code in public repositories that show clear signs of being written by AI has grown continuously since the public release of ChatGPT in November 2022 (Daniotti et al., 2025; OpenAI, 2022).

This raises a structural question. LLMs sit between developers and the software ecosystem as a kind of intermediary, and the temporal boundary of their training data creates an asymmetry in what developers can even learn about through artificial intelligence. Libraries released before the cutoff are visible; those released after it are not. This paper provides an empirical answer in case of the Python package ecosystem.

## 1.2 The Mechanism

The mechanism runs through a property of pretrained language models that is easy to state. Their parametric knowledge is frozen at the end of training. A model whose training data runs through September 2021 knows nothing about a library first released in November 2021, regardless of that library's quality, functionality, or subsequent adoption. When a developer asks such a model to recommend a library for a task, or when the model generates code to solve one, the post-cutoff library is not part of the AI's list of suggestions. The pre-cutoff library is, even if it is older and not as good as the newer one.

This asymmetry is not hypothetical. The GPT-3.5 and GPT-4 model families, which powered the majority of AI coding assistant usage between 2022 and 2024, share a documented training data cutoff of September 2021 (OpenAI, 2023). A Python library released before October 2021 was included in these models' knowledge, and a library released in November 2021 most certainly was not. If LLM recommendations shape adoption, this discrete boundary should produce a detectable discontinuity in the diffusion trajectories of libraries released just before and just after that date.

A plausible causal chain links these stages. Training cutoffs create differential knowledge

[1] AI tools were used to assist with prose review, formatting, and logical consistency across all chapters of this paper. All content was written by the author. See Appendix A for the full declaration.

representation, that representation may feed into recommendation frequencies, and those recommendations may in turn shape developer adoption. The empirical challenge is whether the cumulative effect of these steps is large enough to show up as a noticeable, measurable difference when looking at the entirety of libraries.

## 1.3 The Research Gap

Recent research asks how AI systems shape the diversity of creative output. Doshi and Hauser (2024) run a controlled experiment and show that generative AI lifts the average quality of individual writing while at the same time compressing variation across participants. Individuals assisted by AI tend to produce similar high-quality solutions, and the collective exploration decreases. The implication is that tools which raise individual productivity can quietly narrow the aggregate variety of ideas, products, and approaches a community produces.

The software world is a good place to see if this same logic applies to how people choose tools. If LLMs steer developers toward pre-cutoff libraries, the active tool set may narrow over time, not because post-cutoff libraries are worse, but because they are invisible to the intermediary which developers now use all the time to utilize libraries. That is what I mean by a Knowledge Wall. It is a concentration mechanism that is essentially invisible at the level of any single developer and only shows up in the aggregate distribution of libraries in use.

The empirical literature on technology diffusion and platform economics has documented at length how recommendation systems, network effects, and algorithmic curation influence adoption (Bass, 1969; Rogers, 2003; Kleinberg and Raghavan, 2021; Bommasani et al., 2022a). It has not yet addressed LLM training boundaries as a specific mechanism. The software ecosystem literature documents what makes libraries survive and diffuse (Borges et al., 2016; Borges and Valente, 2018), but predates the mass deployment of AI coding assistants. This study joins these two ideas and provides an empirically identified estimate of how things are working as of now.

## 1.4 Research Setting

The Python Package Index (PyPI) is an unusually well-suited setting for this question. Python is the dominant language for data science, machine learning, and scientific computing (Stack Overflow, 2024), exactly the domains in which AI-assisted coding is most common (Daniotti et al., 2025). PyPI records weekly download counts for every registered package, which makes it possible to construct a diffusion curve for each library starting from the week of its first observed download. GitHub supplies the complementary implementation-level evidence in the form of a panel of weekly import counts per library, with an additional layer of commit-level AI scoring that flags whether each importing commit was generated with LLM assistance (Daniotti et al., 2025).

Together the two sources let me ask the question at two levels. Does the training cutoff affect general developer adoption, as measured by PyPI downloads? And does it specifically affect AI-mediated code generation, as measured by GitHub imports attributable to AI-assisted commits? If the effect is larger in the second, that is evidence the channel runs through LLM-generated code rather than through general attention or discoverability. Because the cutoff date is documented and fixed in September 2021, the design can rely on a standard Regression Discontinuity Design around that boundary.

## 1.5 Research Questions and Hypotheses

The central question is: Do Python libraries released shortly before the GPT-3.5/GPT-4 training cutoff diffuse more widely in the AI-era than those released shortly after?

Four hypotheses structure the analysis. H1: Pre-cutoff libraries show greater post-AI diffusion than post-cutoff libraries. H2: The discontinuity is present in GitHub code imports, consistent with an LLM-mediated code generation channel. H3: The gap emerged only after the public release of ChatGPT (November, 2022). H4: Placebo cutoffs in 2018–2020 do not produce comparable discontinuities.

## 1.6 Design Overview

The value of this setting comes from the natural experiment it provides. OpenAI's data collection schedule set the September 2021 training cutoff, and that schedule is exogenous to any individual library's release decision. I can therefore treat libraries released just before and just after the cutoff as assigned to treatment and control as-if randomly (though the design approaches, rather than achieves, experimental conditions), and use the resulting comparison to estimate the relationship between inclusion in AI training data and adoption of the libraries later on.

The primary estimation strategy is a Difference-in-Discontinuities (Diff-in-RDD) design (Grembi et al., 2016). The 2021 cohort is compared against 2018–2020 placebo years to absorb seasonal patterns. An asymmetric donut exclusion handles delays in how the data was collected and processed near the boundary, and the success tiers are defined using pre-ChatGPT performance so that no post-treatment outcome information is in the sample filter. Figure 1.1 illustrates the design below.

## 1.7 Preview of Findings

The main finding is a widening gap in weekly PyPI downloads at the September 2021 cutoff. Before ChatGPT, libraries on either side of it differed by only about 28%, roughly the autumn

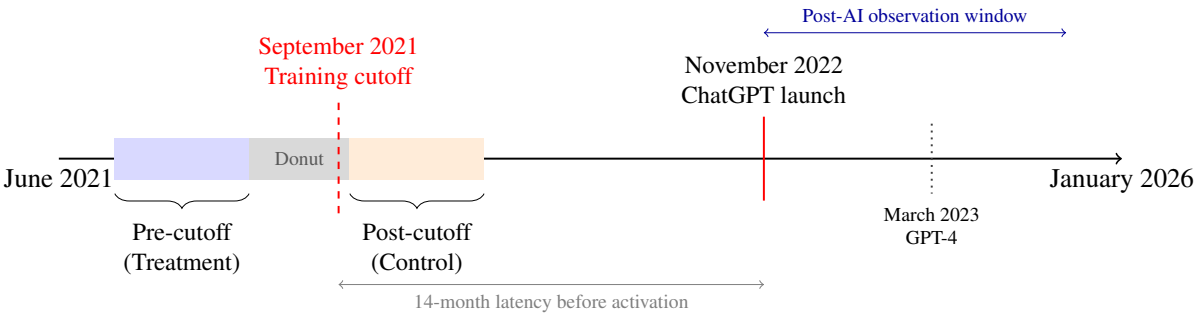


Figure 1.1 - Timeline of the natural experiment. The exogenously defined September 2021 training data cutoff assigns libraries released before the boundary to the treatment group (included in training data) and those released after to the control group (excluded), as-if randomly. The 9-week donut excludes libraries with ambiguous training inclusion. The Knowledge Wall effect activates only after November 2022, when ChatGPT provides a mass interface through which the training boundary becomes consequential for developer behavior. *Source:* Author's own elaboration.

release premium. After ChatGPT, pre-cutoff libraries pull away. By early 2026 they average roughly five times the weekly downloads of libraries released just a few weeks later, and there is no sign that the gap has stopped growing. The baseline-adjusted Difference-in-Discontinuities design, using 2018–2020 as placebo years, confirms that this cannot be explained by seasonal patterns or pre-existing quality differences ($p = 0.002$). Restricting the sample to libraries that had already shown pre-ChatGPT quality leaves the gap essentially unchanged, which rules out low-quality noise at the boundary. A Diff-in-RDD on the GitHub-matched subsample finds a directionally larger excess jump for code imports than for downloads, consistent with LLM-steered code generation as the channel, though the matched sample is too small for the estimate to be significant on usual levels. None of this was visible before November 2022; the gap opens only once ChatGPT is publicly released.

## 1.8 Contribution

To the best of my knowledge, this is the first quasi-experimental estimate of the causal effect of LLM training data boundaries on the diffusion of new software packages. Identification relies on a documented institutional feature, the dated GPT-3.5/GPT-4 training data cutoff, to generate a discontinuity in what we expect the AI to know. I also use a setup that looks at two different outcomes to get a clearer picture of what is happening, that separates general developer adoption (PyPI) from implementation-level usage (GitHub). This makes it possible to isolate a channel effect that raw download data cannot show on its own. Beyond the empirical contribution, the paper gives concrete evidence for the argument articulated by Doshi and Hauser (2024) in the context of creative output, namely that AI systems which improve individual productivity can at the same time narrow collective adoption diversity. In this paper, the mechanism through which that narrowing operates is the boundary of the model's training data specifically.

## 1.9 Paper Structure

Chapter 2 reviews the relevant literature across several bodies of work, including AI and collective diversity, algorithmic monoculture, technology diffusion theory, and LLM knowledge boundaries. Chapter 3 describes the data sources and sample construction. Chapter 4 formalizes the empirical design. Chapter 5 presents the results. Chapter 6 interprets the findings and discusses limitations. A short chapter concludes.

# Chapter 2 – Literature Review

This chapter situates the paper within several different academic topics. Section 2.1 covers the emerging literature on AI and collective diversity. Section 2.2 reviews research on algorithmic monoculture and platform concentration. Section 2.3 develops the theoretical context from technology diffusion and innovation ecology. Section 2.4 surveys empirical work on LLMs in economic activity. Section 2.5 covers more technical work on LLM knowledge boundaries and their practical limits. Section 2.6 states what the paper adds.

## 2.1 AI, Creativity, and Collective Diversity

The central theoretical concern motivating this paper is that tools which improve individual productivity may at the same time narrow the collective variety of what they help produce. The clearest empirical evidence for this idea is Doshi and Hauser (2024). They run a preregistered online experiment in which participants write short stories with or without access to a generative AI system. Access to AI raises average story quality, which is in line with individual productivity gains. The same access also decreases the variation across stories; AI-assisted writing turns out to be significantly more uniform than the purely human work. The authors interpret this as a social dilemma. Each individual writer is better off with AI assistance, yet the aggregate range of novel content the group produces is narrower. The mechanism behind that pattern is convergence. AI suggestions push participants toward similar high-quality solutions and leave no room for the different, creative ideas people usually come up with on their own.

The analogy to software adoption is structural rather than empirical. Doshi and Hauser (2024) study creative text; this paper studies tool selection. The underlying logic is the same. When a developer queries an LLM, the model relies on its parametric knowledge to suggest options. When many developers prompt the same model, they receive correlated suggestions, all concentrated on libraries represented in training data. The collective effect is a quiet narrowing of the active tool set, not because post-cutoff libraries are worse, but because the information broker through which most developers now access the ecosystem cannot consider them. Doshi and Hauser (2024) serve rather as motivation and structural analogy here, and not directly comparable empirical evidence.

Related experimental work strengthens that the basic logic of this process is believable. Anderson et al. (2024) study creative ideation in an experiment where participants use either ChatGPT or an alternative support tool. Users of ChatGPT generate more detailed ideas, but conceptually more overlapping ones, than users of the alternative. The homogenization is group-level rather than individual. Different users receive similar suggestions from the same model and converge on similar ideas even when working independently. That kind of group-level convergence is precisely the mechanism that would, if it carried over to library discovery, produce the

population-level pattern this paper identifies. A closely related phenomenon is described by Wu et al. (2025) as generative monoculture. Their work shows that LLM outputs are narrower in diversity than the training sets they come from. Rather than saying this happens because of the data itself, they trace it to reinforcement learning from human feedback (RLHF). The implication is that even with updated training data, the pressure toward uniformity would persist, since the process itself rewards the outputs with the most probability.

Raghavan (2024) offers a theoretical counterpoint with a game-theoretic model, showing that competition among AI providers can select for diverse outputs at equilibrium and reduce the monoculture Wu et al. identify. The empirical implication is that this trend toward similarity might weaken as the market for AI tools becomes more diverse. The competitive equilibrium result, however, applies to markets with genuinely differentiated providers. It does not address the case studied here, where the dominant models (GPT-3.5 and GPT-4) shared a common training cutoff during the period of mass deployment, and therefore a common knowledge boundary. So competition may moderate monoculture in outputs, it does not eliminate a shared temporal constraint in training data.

## 2.2 Algorithmic Monoculture and Platform Concentration

The broader literature on algorithmic monoculture addresses the systemic risks that arise when shared decision systems are rolled out to a wide audience. Kleinberg and Raghavan (2021) formalize the core argument in a probabilistic framework. When multiple decision-makers adopt the same algorithm, the correlation in their decisions reduces collective decision quality, even if the algorithm is individually optimal, because correlated errors cannot be diversified away. Applied to library discovery, when many developers prompt the same LLM, the different parts of the Python library universe they look at is correlated. Adoption concentrates on the same subset of pre-cutoff tools and the effective diversity of solutions shrinks.

Bommasani et al. (2022a) extend this framework to foundation models[1] specifically. They test whether shared model components make outcomes similar across decision-makers in vision and language tasks, and find that sharing training data consistently makes different people's results look more and more alike, with effects that exceed group-level averages. Their result is directly relevant to the mechanism this paper proposes. When developers share the same base model, correlation in library suggestions is an empirical prediction of the framework rather than just a guess about what might happen.

The broader infrastructure risks of foundation model concentration are cataloged in Bommasani et al. (2022b), a long technical report on the societal implications of models trained on broad corpora and adapted to other tasks that the model is used for later at scale. The report identifies homogenization as a first-order risk, since concentrated model infrastructure creates

[1]Foundation models are large-scale models trained on broad data and adapted to a wide range of downstream tasks. Examples include GPT-4, Claude, and Gemini. The term was introduced by Bommasani et al. (2022b).

correlated failure modes across applications. The Knowledge Wall described in this paper is one specific instance of that general risk, operating through the time limit on the AI's knowledge, rather than just how widely the AI is used.

## 2.3 Technology Diffusion and Innovation Ecology

The study of how new technologies spread through populations has a long lineage in economics and sociology. Bass (1969) established the foundational diffusion model and decomposed adoption into two components, innovators who adopt independently of social influence and imitators who are influenced by the number of prior adopters. In the present-day software ecosystem, LLMs have entered this adoption process as a new type of information intermediary, affecting the way developers find and use libraries that are already popular. For a library to reach imitators through LLM-mediated discovery, it must first be represented in the model's parametric knowledge. Libraries outside the training corpus drop out of the LLM recommendation channel and lose access to a large share of the imitator population. The Knowledge Wall is, in this sense, a structural modification to the Bass imitation process. It blocks one specific route through which new adopters would otherwise hear about a library.

Rogers (2003) characterized adopter categories from innovators through laggards and identified observability as a key attribute accelerating adoption. Potential adopters are more likely to take up a technology they can see in use. Observability is directly constrained by the Knowledge Wall. A library that the dominant LLM cannot recommend is less likely to appear in LLM-generated code, and less visible to developers who rely on AI-assisted workflows. Rogers's framework implies that reduced observability has first-order effects on adoption rates, especially in the early majority phase where seeing others use a tool makes it spread even faster.

The general-purpose technology framework of Bresnahan and Trajtenberg (1995) characterizes transformative technologies by three properties: pervasiveness, ongoing improvement, and innovational complementarities, meaning that the technology encourages new ideas and product development in the industries that use it. Eloundou et al. (2024) confirm that LLMs satisfy these criteria and function as a general-purpose technology in this sense. The Bresnahan and Trajtenberg framework also implies a specific failure mode. When a general-purpose technology's capabilities are temporally bounded, its innovational complementarities depend on which tools and technologies were already available in the past. Post-cutoff libraries are excluded from the LLM's complementary network at the point of mass deployment, and the technology's complementarity benefits concentrate on the pre-cutoff tool set.

Weitzman (1998) argues that economic growth proceeds through combinatorial recombination of existing ideas, and that the pace of innovation depends in part on the variety of components available for recombination. The Knowledge Wall restricts that process by narrowing the menu of components the dominant discovery mechanism can present. If developers encounter only pre-cutoff libraries through LLM-mediated search, the combinatorial space available for

software development gradually converges toward the pre-cutoff universe. Arthur (2009) extends the same logic to technological evolution and argues that new technologies are assembled from existing ones, so the available toolkit determines what combinations are feasible at all. A frozen LLM knowledge base then acts as a constraint on the combinatorial toolkit available to developers who rely primarily on AI models to guide them to new libraries.

Azoulay et al. (2019) provide a remarkable empirical analogue from academic science. The premature death of eminent scientists releases their fields from incumbency effects and lets outsiders introduce new, highly cited ideas that would otherwise have been crowded out. The mechanism is suppression of entry by established figures, regardless of the quality of the entrants. The dynamic is structurally analogous to the Knowledge Wall. Pre-cutoff libraries function as incumbents in the LLM's parametric knowledge, and post-cutoff libraries are would-be entrants suppressed not by intentional gatekeeping but by how the platform that connects users and data is designed. The key parallel is that the entry suppression in Azoulay et al.'s work is not driven by quality differences between incumbents and entrants, just as the Knowledge Wall is not driven by quality differences between pre- and post-cutoff libraries.

## 2.4 LLMs in Economic Activity

The empirical economics of LLM adoption is a fast-moving field. Eloundou et al. (2024) estimate that roughly 80% of US workers have at least 10% of their tasks exposed to LLMs, with software development among the professions that use these tools the most. That exposure number sets the demand-side context for the Knowledge Wall. If most developers are now LLM-assisted, then the training cutoff boundary touches a good portion of technology adoption decisions in the population.

Direct causal evidence on productivity effects comes from randomized experiments. Noy and Zhang (2023) assign college-educated professionals to writing tasks with or without ChatGPT access and find roughly a 40% reduction in completion time and an 18% improvement in evaluator ratings. The effects are largest for lower-ability workers, a a shrinking variety that is consistent with Doshi and Hauser (2024). Brynjolfsson et al. (2023) study a deployed AI assistant in a customer service setting and find a 14% average productivity gain, with a 34% gain for novice workers. Both studies pick up the same distributional pattern. AI assistance improves performance while compressing variance, and the model effectively distributes the practices of more capable workers to less experienced ones. For library selection, that compression dynamic implies that less experienced developers, who rely hardest on LLM recommendations, are also the ones most exposed to the Knowledge Wall.

For software development specifically, Peng et al. (2023) run a controlled experiment in which developers with GitHub Copilot complete a programming task 55.8% faster than those without. The gain is largest for inexperienced developers. These are exactly the people most likely to accept LLM library suggestions without independent evaluation, and therefore the

population through which the Knowledge Wall has the biggest impact on how quickly new software is picked up.

Daniotti et al. (2025) provide large-scale observational evidence on the diffusion of AI coding tools in the Python ecosystem. They train a neural classifier to identify AI-generated functions across 80 million GitHub commits by 200,000 developers between 2019 and 2024. Their adoption trajectory shows sharp increases in AI-generated code coinciding with the launches of GitHub Copilot and ChatGPT, with US developers reaching roughly 30% AI-generated code by December 2024. That figure sets the quantitative scale of LLM mediation in the Python ecosystem that the Knowledge Wall mechanism requires. If something like a third of Python functions in active repositories are AI-generated, the training boundary determines what a substantial share of production code can reference at all. Daniotti et al. also find that AI use is associated with increased exploration of new libraries, including libraries novel to the ecosystem. That result is a useful boundary condition for the present analysis. Aggregate library exploration expands under AI assistance, but the training boundary constrains which libraries enter that expanded set. The Knowledge Wall does not stop people from trying new things; it just pushes them toward tools the AI already knows about.

Acemoglu (2024) provides macroeconomic context for these findings. He estimates the aggregate productivity impact of current AI systems at no more than a 0.66% increase in total factor productivity over a decade, well below the more optimistic projections in the earlier literature. A modest aggregate estimate does not inhibit substantial effects on specific dimensions of behavior, such as library adoption. The Knowledge Wall operates mainly as a distributional mechanism. It redirects adoption from post-cutoff to pre-cutoff tools and changes the composition of the active tool set, while leaving aggregate developer activity roughly unchanged.

## 2.5 LLM Knowledge Boundaries

A prerequisite for the Knowledge Wall mechanism is that LLM knowledge is genuinely and consequentially bounded by training data cutoffs. Several recent empirical studies confirm that this is the case. Cheng et al. (2024) show that self-reported training cutoff dates are often unreliable. The effective knowledge boundary, defined by where model accuracy lessens, frequently differs from the provider-declared date, partly because of temporal biases in CommonCrawl snapshots and deduplication artifacts. Their finding has a direct implication for identification. The true cutoff is not an exact calendar date but a zone of decreasing data coverage, which gives a substantive reason for the 9-week asymmetric donut exclusion used here, since the donut removes libraries whose training inclusion status is uncertain. Pezik et al. (2025) provide complementary measurement through a benchmark of 1,713 factual questions about recent events. They show that model accuracy drops sharply near declared cutoffs, with refusal rates above 95% for questions about events immediately after the cutoff. That degradation gives direct empirical grounding for the assumption that post-cutoff libraries are effectively unavailable

to model recommendations.

Gao et al. (2024) survey the state of retrieval-augmented generation (RAG), the main technical mechanism through which LLMs can in principle access knowledge beyond their training corpus. The survey documents persistent limitations. Retrieval quality is unreliable, latency and cost are substantially higher than for parametric inference (the AI's ability to answer using only the data it was trained on), and long-context retrieval degrades because of attention dilution (the model's focus is spread too thin across too many topics). These limitations matter for one of the main threats to this paper's interpretation. If users routinely combined parametric LLM knowledge with real-time web retrieval, the training boundary should have eroded once RAG became available. The survey evidence is consistent with what the actual usage numbers show, which shows a persistent and widening gap through early 2026, and suggests that parametric knowledge remains the default in most production deployments. Finally, Shan et al. (2024) document opposing forces that work together to balance each other out. They show that content creators can introduce targeted data poisoning attacks to contaminate AI training data and deter future scraping of their work. The existence and adoption of such tools largely implies that the effective knowledge boundary may narrow further as more content is deliberately excluded from future training runs, which reinforces rather than softens the knowledge asymmetry described here.

## 2.6 Contribution of this Paper

The literature reviewed above establishes that the Knowledge Wall is theoretically plausible, but it does not offer a direct empirical test of the mechanism. The AI and diversity literature documents narrowing in creative output rather than technology adoption. The monoculture literature provides theory and experimental evidence on outcome homogenization but has not been applied to the software package ecosystem. The technology diffusion literature provides structural frameworks that predate the mass deployment of LLMs. The AI productivity literature documents individual-level gains but does not look at ecosystem-level distributional consequences, comparing what happened before and after the model's knowledge ends. The LLM boundaries literature confirms that knowledge cutoffs are real and measurable but does not go as far as looking at how this actually changes what people buy or use.

To the best of my knowledge, this paper provides the first quasi-experimental estimate of the causal effect of an LLM training boundary on the diffusion of new software packages. Identification leans on the precisely documented September 2021 knowledge cutoff of the GPT-3.5 and GPT-4 families, with a Regression Discontinuity Design in which the running variable is library release timing relative to that cutoff. A Difference-in-Discontinuities augmentation controls for seasonal patterns in the September release calendar, using three pre-LLM cohorts (2018–2020) as placebo controls. The two-outcome design, comparing PyPI download counts against GitHub code import counts, makes a channel effect identifiable. If the mechanism runs

specifically through LLM-generated code, the discontinuity should be larger in implementation data than in general download data. The pre-AI success tier logic, which conditions estimation on libraries that demonstrated human-assessed quality entirely before ChatGPT, addresses the quality-sorting concern that would otherwise undermine causal interpretation.

Three aspects of the design give a fresh perspective that hasn't been covered in previous studies. First, using a publicly documented training cutoff date as the treatment discontinuity utilizes an institutional feature that laboratory experiments on AI effects do not have access to. Second, combining population-level download data ($N > 500{,}000$ packages) with implementation-level data carrying AI generation signatures lets the mechanism be tested at the scale of the full Python ecosystem. Third, the activation pattern test, which examines whether the gap was absent before November 2022 and emerged only once LLMs reached mass deployment, provides a specific prediction about time that can be proven wrong. That prediction is what distinguishes the LLM knowledge boundary mechanism from generic seasonal or long term explanations.

# Chapter 3 – Data

## 3.1 Overview

The empirical analysis draws on two sources. The first is a panel of weekly download counts from the Python Package Index (PyPI). The second is a weekly panel of GitHub code imports constructed from public repository data. PyPI captures library diffusion in the general developer population, while GitHub captures implementation, writing code that imports a library, and has an AI-scoring layer used in the mechanism test. The two sources are linked by normalized package names.

## 3.2 PyPI Download Data

### 3.2.1 Source and Coverage

The primary data source is a package-week panel of PyPI download counts extracted from Google BigQuery's public PyPI dataset (Python Software Foundation, 2024). The raw file covers all packages observed in the PyPI registry with at least one recorded download week. The data covers a few years up to January 2026, which gives us more than four years to see how libraries released around the September 2021 boundary have spread.

### 3.2.2 Unit of Observation

The raw panel is at the (package, week) level, where weeks are aligned to Monday. The key aggregate unit for analysis is the package, with outcomes defined as cumulative download totals calculated over set periods of time after the release.

### 3.2.3 Release Date Construction

For each package, the release date is the first calendar week in which the package recorded at least one PyPI download. This is identified in a two-pass procedure. Pass 1 scans the panel to find the minimum positive-download week for each package, and Pass 2 aggregates download counts over pre-defined horizons anchored to that release week. Package names are normalized to lowercase with underscores replaced by hyphens before grouping, so that name variants mapping to the same standardized names are merged. When multiple project names normalize to the same package identifier, the earliest observed download week across all variants is used as the release date.

### 3.2.4 Outcome Construction

Five cumulative download outcomes are constructed for each package.

Table 3.1 - PyPI Download Outcome Variables

| Variable | Definition |
|---|---|
| `cum_downloads_26wk` | Cumulative downloads in the first 26 weeks post-release |
| `total_downloads_52wk` | Cumulative downloads in the first 52 weeks post-release |
| `cum_downloads_gpt4` | Cumulative downloads up to and including the week of GPT-4 release (March 13, 2023) |
| `post_ai_downloads_alltime` | Cumulative downloads from November 28, 2022 (ChatGPT public release) onward |
| `cum_downloads_alltime` | Total lifetime downloads through end of panel |

*Source:* Author's compilation based on PyPI data.

The 26-week outcome is used exclusively for the success-tier classification (Section 3.5). The 52-week and post-AI outcomes are the primary estimands.

### 3.2.5 Raw Universe

The full PyPI panel contains 871,133 unique packages after normalization.

## 3.3 GitHub Implementation Data

### 3.3.1 Source and Coverage

The secondary data source is a library-week panel recording weekly import counts per library across a sample of public GitHub repositories. The panel captures the count of distinct code files (or commits) that contain an import statement referencing a given library in a given week. Coverage extends through early 2026.

### 3.3.2 AI Scoring

Each week-library observation in the GitHub panel carries an AI commit score, a bias-corrected point estimate of the proportion of code that is AI-generated. The underlying classifier assigns a probability to each function or code block, and these raw predictions are then aggregated and adjusted for the classifier's estimated false positive and true positive rates. Because the correction subtracts an expected noise floor, the resulting score can turn slightly negative for commits or groups with very low AI adoption, when the raw prediction rate does not exceed the classifier's baseline false positive rate. The score is aggregated to the package level as a commit-weighted average AI score over the first 52 weeks post-release. Packages above the

median are classified as High AI and those below as Low AI. This variable is used only in the exploratory mechanism test (Section 4.9).

### 3.3.3 GitHub Outcome Variables

The following cumulative import outcomes are constructed.

Table 3.2 - GitHub Import Outcome Variables

| Variable | Definition |
|---|---|
| `cum_imports_52wk` | Cumulative imports in the first 52 weeks post-release |
| `cum_imports_gpt4` | Cumulative imports up to March 13, 2023 (GPT-4 release) |
| `post_ai_imports_alltime` | Cumulative imports from November 28, 2022 onward |
| `cum_imports_alltime` | Total lifetime imports through end of panel |

*Source:* Author's compilation based on GitHub data.

### 3.3.4 Linkage to PyPI

GitHub libraries are matched to the PyPI package universe by normalized name, using an exact match after applying the same normalization function used in the PyPI data processing. The match is performed as an inner join, so GitHub observations are kept only for packages that appear in the PyPI release table. Release dates are taken from PyPI, not from GitHub. All post-release aggregation on the GitHub side therefore uses the PyPI-derived release week as the time anchor.

This linkage rule is designed to be safe rather than bold. Aggressive fuzzy matching would slightly inflate the matched sample but also introduce false matches, especially among short or generic package names. While I lose some data, the results are much more accurate this way.

## 3.4 Sample Construction and Attrition

For each cohort year, the analysis sample is constructed through the following steps.

### 3.4.1 Step 1 - Time window restriction

A package is retained if its release week falls within $\pm 156$ weeks (three years) of the cohort cutoff date. Packages outside this window are dropped. The 156-week window is intentionally wide, since bandwidth selection for the RDD estimator is data-driven and will typically pick a narrower effective sample.

### 3.4.2 Step 2 - Minimum download filter

A baseline filter requiring at least 10 cumulative downloads at 52 weeks defines the Broad tier. This filter is applied as a sample restriction in the Broad specifications and is varied in robustness checks.

### 3.4.3 Step 3 - Donut exclusion

Observations with running variable values in $[-8, 0]$, roughly the nine-week period from early August through September 2021, are excluded from all main estimation samples.[1] The exclusion is recorded as a flag on the full dataset and applied at the estimation stage.

The table below reports sample sizes at the construction stage for all cohort years.

Table 3.3 - Sample Sizes by Cohort Year

| Cohort | In-Window (PyPI) | GitHub Matched | GitHub Match Rate |
|---|---|---|---|
| Placebo 2018 | 381,390 | 37,696 | 9.9% |
| Placebo 2019 | 399,000 | 30,204 | 7.6% |
| Placebo 2020 | 482,862 | 30,672 | 6.4% |
| Main 2021 | 543,177 | 28,991 | 5.3% |

*Source:* Author's calculations based on PyPI and GitHub data.

The decline in the GitHub match rate over time reflects growth in the PyPI universe. The number of registered packages has grown substantially since 2018, while GitHub coverage of those packages has not kept up with that. Estimation samples for the main RDD analysis are further reduced by the donut exclusion and by bandwidth selection, and final estimation $N$ values are reported alongside each table in Chapter 5.

### 3.4.4 Missing GitHub Values

Packages not matched to GitHub retain missing values (NaN) for all GitHub-derived variables. These are not imputed to zero, since absence of a GitHub match does not imply zero imports. It only implies no coverage in the GitHub data source. PyPI and GitHub analyses therefore are done on separate, partially overlapping samples. All GitHub-based estimates are conditional on the matched subsample and should be read accordingly.

## 3.5 Success Tiers

A secondary outcome is whether a library crosses a minimum download threshold within its first 26 weeks of release. Three tiers are defined.

[1] The exploratory AI-exposure mechanism split (Appendix F) retains the full bandwidth window because the subsample is already small ($N = 1{,}303$).

Table 3.4 - Success Tier Definitions

| Tier | Threshold | Rationale |
|---|---|---|
| Broad | $\geq$ 10 downloads at 52 weeks | Minimal signal of non-spam adoption |
| Successful | $\geq$ 500 downloads at 26 weeks | Meaningful community foothold |
| Superstar | $\geq$ 1,000 downloads at 26 weeks | High-visibility early adoption |

*Source:* Author's compilation.

The thresholds are measured strictly within the first 26 weeks post-release. For the Main 2021 cohort, the 26-week window closes around March 2022, well before ChatGPT launches on November 28, 2022. The treatment in this design is the public release of ChatGPT, so filtering libraries on outcomes observed before that date does not select on the dependent variable. Tier membership reflects only pre-treatment developer adoption, not any later AI-mediated boost - which is crucial for causal identification (no circulation).

For the placebo cohorts (2018–2020), the 26-week window closes four to seven years before ChatGPT, without the danger of the data being mixed up or influenced by other factors.

## 3.6 Key Milestones and Calendar Reference Points

The following dates are used as fixed reference points throughout the analysis.

Table 3.5 - Key Calendar Reference Points

| Event | Date |
|---|---|
| GPT-3.5 / GPT-4 training data cutoff | September 27, 2021 |
| ChatGPT public release | November 28, 2022 |
| GPT-4 public release | March 13, 2023 |
| GPT-4 Turbo release | November 6, 2023 |
| Placebo cutoff (2018) | September 24, 2018 |
| Placebo cutoff (2019) | September 30, 2019 |
| Placebo cutoff (2020) | September 28, 2020 |

*Source:* Author's compilation based on OpenAI documentation.

Placebo cutoff dates are chosen as the Monday-aligned week nearest to September 27 in each respective year, preserving exact calendar position for comparability across cohorts.

Figure 3.1 displays the density of library release dates relative to the September 2021 cutoff. There is no visible discontinuity at the boundary, supporting the identifying assumption that library developers did not systematically time releases around the LLM knowledge cutoff.

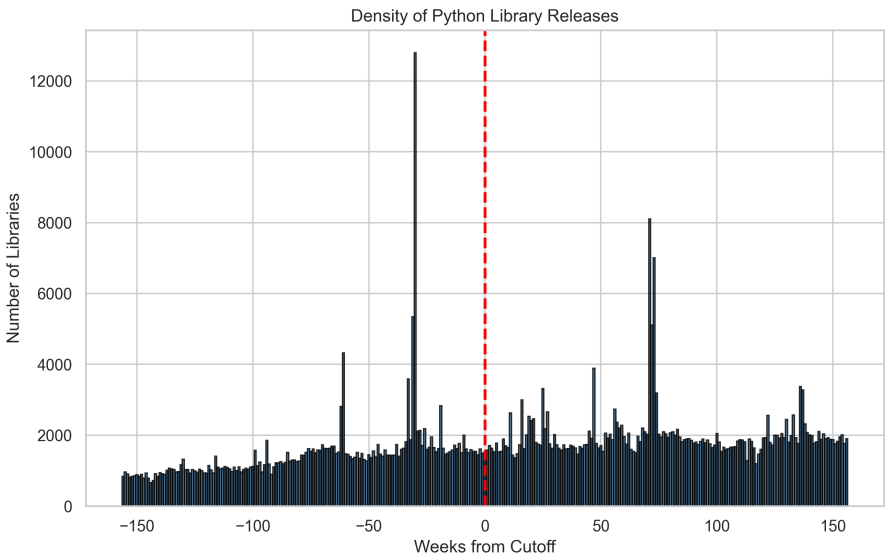


Figure 3.1 - Density of library releases by week relative to the September 2021 cutoff. The dashed red line marks the cutoff. No systematic bunching is evident at the boundary, consistent with the assumption that release timing is not strategically manipulated around the LLM training cutoff. *Source:* Author's calculations based on PyPI data.

## 3.7 Measurement Limitations

### 3.7.1 Release Date Proxy

The first observed download week is a noisy proxy for the actual release date. A package may exist on PyPI, or as a GitHub repository, before generating any downloads. If this pre-observation delay is longer for post-cutoff packages, which is plausible if early adoption is suppressed, the running variable is biased toward zero and the estimated effect is attenuated. This approach is more likely to underestimate the effect than to exaggerate it.

### 3.7.2 PyPI Download Inflation

PyPI download counts include automated CI/CD pipeline pulls, mirror syncs, and bot traffic. Raw counts therefore overstate genuine human adoption. The log transformation softens the influence of extreme values, but the level of any individual package's downloads cannot be read as a direct measure of developer intent. The limitation affects levels, not discontinuities, as long as automated download patterns are smooth across the cutoff.

### 3.7.3 GitHub Coverage and Selection

The GitHub panel covers a non-random subset of the PyPI universe. Libraries with higher visibility, more active communities, and more GitHub-hosted dependent projects are over-

represented. The 5.3% match rate for the 2021 cohort implies that GitHub-based estimates apply to a selected, likely more prominent part of the library population than the PyPI-based estimates do. Generalizing from the GitHub subsample to the full PyPI universe needs to be handled with caution.

### 3.7.4 AI Score Endogeneity

The commit-level AI score used to construct the mechanism moderator is measured over the 52 weeks following release. A library suppressed by the LLM cutoff attracts fewer imports overall, and specifically fewer AI-generated imports, because it is less known to LLMs. The moderator therefore partly reflects treatment itself. This endogeneity concern applies only to the exploratory mechanism test and is acknowledged there. It does not affect the PyPI or GitHub diffusion outcomes, which serve as the logical basis for causal inference.

### 3.7.5 Package Identity

Name normalization collapses variants of the same package name but cannot identify library forks, renames, or namespace transfers. A library substantially renamed after release would have its pre- and post-rename download histories treated as separate packages. The direction of bias from this error is ambiguous and likely small in aggregate.

# Chapter 4 – Empirical Design

## 4.1 Overview

This paper uses a Regression Discontinuity Design (RDD; Imbens and Lemieux, 2008; Lee and Lemieux, 2010; Cattaneo et al., 2020a; Cattaneo and Titiunik, 2022) to identify whether Python libraries released shortly before the September 2021 LLM training cutoff diffuse more widely than libraries released shortly after it. The core identifying assumption is that, absent the cutoff, the expected diffusion trajectory of libraries released just before and just after the boundary would have been continuous. Any discrete jump at the cutoff is then attributed to how the LLM knows more about some topics than others.

Because the cutoff operates in calendar time rather than in a cross-sectional threshold, the setting is a regression discontinuity in time (Hausman and Rapson, 2018). The baseline design is augmented with a Difference-in-Discontinuities (Diff-in-RDD) strategy (Grembi et al., 2016; Shinoki et al., 2024) that uses three pre-LLM release cohorts (2018–2020) as placebo controls, netting out seasonal variation and any consistent, multi-year trend in when software is released in September. A mechanism test based on AI-scored GitHub commits is also included as an exploratory extension.

## 4.2 Running Variable and Cutoff Date

The running variable $X_i$ is the release timing of library $i$, measured in weeks relative to the LLM training cutoff. The cutoff date used is September 27, 2021, the approximate knowledge cutoff for the GPT-3.5 and GPT-4 model families (OpenAI, 2023). Libraries released before that date are in principle in the pretrained knowledge of these models, and libraries released after it are not.

Release timing is operationalized as the first calendar week in which a library recorded at least one PyPI download. This definition is preferred over metadata-reported release dates for three reasons. First, metadata dates are frequently missing or incorrect in the PyPI registry. Second, the first observed download week is reproducible from the download panel and requires no external data source. Third, conceptually, the first download week is the right anchor, because it marks the moment a library becomes observable to the developer community.

The running variable is expressed as $X_i = w_i - c$, where $w_i$ is the first observed download week of library $i$ and $c$ is the week containing September 27, 2021. By construction, $X_i < 0$ for pre-cutoff libraries and $X_i \geq 0$ for post-cutoff libraries.

### 4.2.1 Measurement threats

Three threats to running variable validity are worth flagging.

#### Heaping

If developers systematically time releases around calendar milestones, the density of $X_i$ may show non-smooth bunching (Barreca et al., 2016). Density tests based on McCrary (2008) and Cattaneo et al. (2020b) are reported.

#### Delayed observation

A library may exist before its first download, which causes $X_i$ to overstate recency. If post-cutoff libraries are suppressed early, this creates a conservative bias toward zero.

#### Ambiguous inclusion

The exact training data boundary is uncertain. The donut exclusion described in Section 4.4 handles this directly.

## 4.3 Treatment Assignment and Estimand

Library $i$ is assigned to the post-cutoff group ($D_i = 1$) if $X_i \geq 0$ and to the pre-cutoff group ($D_i = 0$) if $X_i < 0$.

The quantity of interest is the Local Average Treatment Effect at the cutoff (LATE; Hahn et al., 2001),

$$\tau = \lim_{x \downarrow 0} \mathbb{E}[Y_i \mid X_i = x] - \lim_{x \uparrow 0} \mathbb{E}[Y_i \mid X_i = x],$$

where $Y_i$ is a measure of library diffusion.

## 4.4 Donut Exclusion

Libraries released within roughly eight weeks before the cutoff are in an ambiguous zone, partly because of data pipeline lags and crawl schedules. The primary specification applies an asymmetric 9-week donut exclusion that drops libraries with $X_i \in [-8, 0]$.

The donut is asymmetric because the post-cutoff boundary is definitionally clean. Any library released after (the week of) September 27, 2021 is excluded from the training corpus by construction. As a robustness check, a symmetric $\pm 4$-week donut is also estimated.

## 4.5 Outcome Variables

### 4.5.1 PyPI downloads

The primary outcome is the cumulative count of PyPI downloads, measured in $\log(1 + \text{downloads})$. Three horizons are used.

#### 52-week cumulative downloads

Primary long-run outcome.

#### Post-AI cumulative downloads

All downloads from November 28, 2022 onward. This horizon isolates the activation window after ChatGPT became a mass interface.

#### 26-week cumulative downloads

Used for the covariate balance analysis.

The 52-week outcome is *horizon-normalized*, in the sense that every library is measured over the same relative window regardless of release date. The post-AI outcome is instead anchored in *calendar time*, since it captures all downloads since a fixed date, so the accumulation window grows as the data extend. This has two implications. The post-AI estimate is a snapshot, and would yield a different (likely larger) coefficient if re-estimated with data collected six months later. In addition, libraries released at different distances from the cutoff have been alive for different durations when the post-AI window opens, which introduces a smooth age gradient. The local linear specification absorbs that gradient through its slope terms, so the discontinuity estimate captures only the discrete jump at the cutoff, not the continuous age difference. The two outcome types are complementary. The horizon-normalized measure ensures comparability, and the calendar-anchored measure directly tests whether the gap activates after LLMs become widely deployed.

### 4.5.2 GitHub imports

A secondary outcome is the cumulative count of GitHub code files importing library $i$, also transformed as $\log(1 + \text{imports})$. The implementation gap, comparing the discontinuity in GitHub imports against the discontinuity in PyPI downloads on the matched subsample, serves as an exploratory channel test.

## 4.6 Local Linear RDD Estimator

The main estimating equation is a local linear regression on each side of the cutoff,

$$Y_i = \alpha + \tau D_i + \beta_1 X_i + \beta_2 D_i \cdot X_i + \varepsilon_i,$$

estimated on observations within a bandwidth $h$ ($|X_i| \leq h$). The primary specification includes pre-AI 52-week cumulative downloads $\log(1 + B_i)$ as a covariate via the `covs` parameter of `rdrobust`, following the covariate-adjusted RDD framework of Calonico et al. (2019). This ANCOVA-style adjustment absorbs cross-sectional variation in library scale and improves precision without altering the local randomization logic. The covariate is dropped when the outcome is 52-week downloads itself, to avoid circularity.

Estimates use the `rdrobust` package (Calonico et al., 2014, 2020). I use a local linear specification ($p = 1$), following the recommendation of Gelman and Imbens (2019) against high-order polynomial specifications. The bandwidth is fixed at $h = 13$ weeks to match the Diff-in-RDD specification, and stability across wider bandwidths is verified in Section 4.10.1. Inference relies on robust bias-corrected (rbc) confidence intervals (Calonico et al., 2022) and a triangular kernel (Fan and Gijbels, 1996).

## 4.7 Identification with Difference-in-Discontinuities

The main RDD identifies $\tau$ under the assumption that counterfactual diffusion is continuous at the cutoff. The main threat to that assumption is *calendar confounding*. The September window may differ from other months in ways that affect diffusion independently of LLM cutoffs, for example through seasonal variation in developer activity or through a concentration of high-quality releases in the late summer academic cycle. To address this, the primary causal specification is a Difference-in-Discontinuities (Diff-in-RDD) design (Grembi et al., 2016).

Three cohorts of libraries, released in 2018, 2019, and 2020, before any LLM with documented training cutoffs was publicly deployed, serve as placebo controls. The Diff-in-RDD estimator extracts the 2021-specific excess jump,

$$\hat{\tau}^{\text{Diff-in-RDD}} = \hat{\tau}_{2021} - \frac{1}{3} \sum_{t \in \{2018,2019,2020\}} \hat{\tau}_t,$$

where $\hat{\tau}_t$ is the estimated RDD discontinuity for cohort $t$, each estimated at the same calendar position (the Monday-aligned week nearest September 27 in year $t$).

### 4.7.1 Implementation

The Diff-in-RDD is estimated as a stacked weighted least squares (WLS) regression. Pooling all four cohort years, the specification is

$$\log(1+Y_{it}) = \alpha + \beta_0 \log(1+B_i) + \beta_1 D_{it} + \beta_2 \mathbb{1}_{2021} + \beta_3 (D_{it} \times \mathbb{1}_{2021}) + g(X_{it}, D_{it}, \mathbb{1}_{2021}) + \varepsilon_{it},$$

where $B_i$ is the library's pre-AI 52-week cumulative download count, $D_{it} = \mathbb{1}(X_{it} \geq 0)$ is the post-cutoff indicator, $\mathbb{1}_{2021}$ is an indicator for the 2021 cohort, and $g(\cdot)$ contains the running variable $X_{it}$ together with its full set of interactions with $D_{it}$ and $\mathbb{1}_{2021}$, so that each cohort has its own slope on each side of the cutoff. The baseline covariate $\log(1+B_i)$ enters additively. It controls for pre-existing differences in library scale without interacting with the treatment or year indicators, which reduces residual variance and soaks up cross-sectional heterogeneity. When the outcome variable is $B_i$ itself (52-week downloads), the covariate is dropped to avoid circularity. Triangular kernel weights $w_i = 1 - |X_i|/h$ are applied, with bandwidth $h = 13$ weeks. The coefficient of interest is $\beta_3$, the excess discontinuity in the 2021 cohort relative to the placebo average.

Standard errors are clustered at the *year-by-week* level, so each unique combination of cohort year and running variable value forms a cluster. With four cohort years and roughly 18 effective running variable values within bandwidth after donut exclusion, this gives about 72 clusters, which is enough mass for cluster-robust inference.

### 4.7.2 Identifying Assumption

The Diff-in-RDD identifies the causal effect under the assumption that the 2021 discontinuity, conditional on the 2018–2020 baseline discontinuity, would be zero in the absence of the LLM training cutoff. This is weaker than the standard RDD continuity assumption. It does not require September to be identical to other months, only that any September-specific pattern is stable across years. The validity of the assumption is supported by the fact that the estimated placebo discontinuities in 2018–2020 are individually small and jointly insignificant.

## 4.8 Success Tiers and the Barrier-to-Entry Test

To test whether the Knowledge Wall acts as a barrier to entry for high-potential libraries, I report estimates across three success tiers. The *Broad tier* includes all packages with at least 10 cumulative downloads in their first 52 weeks, filtering out the most obvious spam or abandoned toy libraries. The *Successful tier* identifies libraries that achieved a meaningful foothold (at least 500 downloads) within their first 26 weeks; for the 2021 cohort, this window closes before ChatGPT, making this a measure of pre-AI human merit. Finally, the *Superstar tier* captures the right tail of the adoption distribution, representing libraries with high-visibility early adoption

(at least 1,000 downloads). The barrier-to-entry test uses a Linear Probability Model (LPM) to estimate the discontinuity in the probability of reaching the Successful tier.

## 4.9 AI Exposure Moderation

A direct mechanism test asks whether suppression is heterogeneous by AI exposure intensity,

$$Y_i = \alpha + \tau D_i + \gamma \cdot \text{HighAI}_i + \delta \cdot (D_i \times \text{HighAI}_i) + f(X_i) + \varepsilon_i.$$

The coefficient of interest is $\delta$, the differential suppression effect for libraries frequently found in AI-generated code.

## 4.10 Robustness Checks

### 4.10.1 Bandwidth Sensitivity

Estimate stability is evaluated across a range of bandwidths $h \in \{10, 13, 18, 26, 39, 52\}$ weeks. The single-year 2021 RDD turned out to be sensitive to bandwidth choice, and the MSE-optimal bandwidth selector (Imbens and Kalyanaraman, 2012; Calonico et al., 2020) in `rdrobust` failed numerically for the large-$N$ PyPI specifications.[1] That sensitivity is part of the empirical justification for the Diff-in-RDD strategy, which remains stable and significant across the same bandwidth choices by leveraging previous years (2018–2020) as a counterfactual for seasonal release patterns.

### 4.10.2 Donut Specification

The primary design uses an asymmetric 9-week donut, excluding the period from early August to September 27, 2021. The motivation is substantive. There is genuine uncertainty about whether libraries released in the final weeks before the cutoff were successfully scraped and processed into the model's training set. To make sure the result is not an artifact of that specific exclusion, a standard symmetric $\pm 4$-week donut is also reported. The post-AI download discontinuity for the Broad tier remains negative and highly significant (Estimate $= -4.35$, $p < 0.001$), so the Knowledge Wall is not driven by the choice of exclusion window.

[1] For numerical stability the estimation wrapper (`scripts/utils.py`) records any `rdrobust` robust point estimate with absolute value greater than 500 log points as missing, on the grounds that values of that magnitude on a $\log(1+Y)$ scale reflect ill-conditioned bias correction rather than a genuine effect. No estimate reported in the thesis was discarded by this guard; it triggered only on a small number of large-$N$ specifications that also failed bandwidth selection.

### 4.10.3 Placebo Cutoffs

The RDD is replicated for the 2018, 2019, and 2020 cohorts using the same relative calendar dates. None of these placebo years contain a major LLM training cutoff. The absence of significant discontinuities in those years helps rule out alternative explanations such as annual seasonality in Python library releases, for example end-of-quarter or pre-conference release spikes, that could otherwise be mistaken for a treatment effect.

### 4.10.4 Density Test

A core RDD assumption is that there is no sorting or manipulation of the running variable around the cutoff. It is implausible that developers in mid-2021 could have anticipated the exact September 27 training cutoff of models like GPT-4 or the subsequent mass adoption of ChatGPT. Density plots confirm a smooth distribution of library releases across the threshold (Figure 3.1), with no evidence of clustering on the pre-cutoff side. A formal McCrary-style density test (McCrary, 2008; Cattaneo et al., 2020b), implemented as a local linear regression of log bin counts with separate slopes on each side of the cutoff, yields no significant discontinuity at either bandwidth ($h = 26$: $\hat{\theta} = +0.004$, $p = 0.969$; $h = 13$: $\hat{\theta} = +0.080$, $p = 0.432$). In both cases the direction is positive, meaning slightly more libraries are released after the cutoff, which is the opposite of what strategic pre-cutoff timing would produce.

### 4.10.5 Covariate Balance

Pre-treatment covariate balance is assessed using the pre-AI success rate RDD reported in Section 5.5. If libraries on either side of the cutoff are comparable in human-evaluated quality before ChatGPT, then quality sorting cannot explain the post-AI discontinuity.

## 4.11 Permutation Inference

A moving-cutoff permutation test re-estimates the primary RDD at 157 alternative placebo Monday dates between January 2019 and January 2022, yielding an empirical null distribution for the raw 52-week download discontinuity. The methodology and results are reported in Appendix B.

# Chapter 5 – Results

## 5.1 Sequence of Evidence

The empirical evidence is presented in five steps. First, the raw discontinuity in the 2021 cohort shows that a jump exists at the September 2021 boundary. Second, the Difference-in-Discontinuities design shows that the 2021 jump is anomalously small relative to historical norms, which amounts to a net suppression. Third, the barrier-to-entry analysis shows that the suppression operates at the level of library emergence, not only at the level of diffusion levels. Fourth, the implementation gap analysis tests whether suppression is directionally larger in AI-generated code than in general downloads. Fifth, the mechanism moderation test and bandwidth sensitivity analysis fill out the robustness picture.

Throughout, descriptive comparisons come before formal estimates, and magnitude interpretations sit alongside the significance tests.

## 5.2 Descriptive Patterns

Before estimating discontinuities, it is worth characterizing the distribution of the main outcomes. Table 5.1 reports summary statistics for the primary outcome variables in the 2021 cohort.

Table 5.1 - Descriptive Statistics for the 2021 Cohort

| **Variable** | **N** | **Mean** | **Median** | **SD** | **p10** | **p90** |
|---|---|---|---|---|---|---|
| *PyPI Downloads (Broad tier, min 10)* | | | | | | |
| 52-week downloads | 527,361 | 25,661 | 2,307 | 1,439,110 | 96 | 15,456 |
| Post-AI downloads | 527,361 | 748,094 | 4,005 | 49,081,592 | 11 | 44,113 |
| *GitHub Imports (matched subsample)* | | | | | | |
| 52-week imports | 28,243 | 147.1 | 2 | 5,077.0 | 0 | 98 |
| Post-AI imports | 28,243 | 318.4 | 7 | 6,721.7 | 0 | 196 |
| *AI Exposure Score (matched subsample)* | | | | | | |
| avg_ai_score_52wk | 8,566 | 0.272 | 0.232 | 0.391 | $-0.217$ | 0.878 |

*Notes:* Statistics computed on the full 2021 analysis sample within $h = 13$ weeks of the cutoff, excluding the 9-week donut. Downloads and imports are in levels (not logged). AI exposure scores are available only for the GitHub-matched subsample with non-zero commit activity. The score is a bias-corrected estimate and is not bounded to $[0, 1]$; negative values arise when the raw classification rate falls below the classifier's estimated false positive rate.
*Source:* Author's calculations based on PyPI and GitHub data.

The distribution of 52-week cumulative PyPI downloads is heavily right-skewed. The median library in the Broad tier records substantially fewer downloads than the mean, and the

top decile accounts for the majority of total download volume. After the $\log(1 + \text{downloads})$ transformation, the distribution is roughly unimodal. GitHub import counts are even more concentrated. Most matched libraries record zero or near-zero AI-era imports, while a small number of high-visibility libraries account for most of the observed import mass.

Among libraries released within $\pm 13$ weeks of the September 2021 cutoff, donut excluded, the unconditional mean of post-AI cumulative downloads is higher for post-cutoff libraries than for pre-cutoff libraries. That raw gap reflects a seasonal pattern common to all cohort years. Post-September libraries in general accumulate more downloads in subsequent years, for reasons unrelated to LLM knowledge cutoffs. Identifying the cutoff-specific effect therefore requires netting out this baseline, which is what the Diff-in-RDD design does.

## 5.3 Raw Discontinuity at the 2021 Cutoff

Table 5.2 reports the baseline RDD estimates for the 2021 cohort, estimated using `rdrobust` with bias-corrected local linear regression, triangular kernel, and $h = 13$ weeks. The donut exclusion (weeks $-8$ to 0) is applied.

Table 5.2 - Main RDD Estimates, 2021 Cohort

| Tier | Outcome | Estimate | SE | p-value | N |
|---|---|---|---|---|---|
| Broad (min 10) | 52-week Downloads | 9.572 | 2.670 | <0.001 | 527,361 |
| Broad (min 10) | Post-AI Downloads | 13.103 | 4.494 | 0.004 | 527,361 |
| Successful (min 500) | Post-AI Downloads | −0.195 | 3.125 | 0.950 | 407,110 |

*Notes:* Bias-corrected robust estimates via `rdrobust`. Triangular kernel, $h = 13$ weeks, 9-week asymmetric donut. All estimates are on the $\log(1 + Y)$ scale. Post-AI downloads are cumulative from November 28, 2022 onward. The baseline-adjusted specification (controlling for $\log(1 + \text{52-week downloads})$) yields qualitatively similar estimates; the Diff-in-RDD in Table 5.3 reports both.
*Source:* Author's calculations based on PyPI data.

A statistically significant positive discontinuity shows up in the Broad tier. Post-cutoff libraries acquire more downloads than pre-cutoff libraries at the boundary, both at the 52-week horizon ($+9.6$, $p < 0.001$) and in the post-AI activation window ($+13.1$, $p = 0.004$). The positive sign reflects the direction of the raw jump. Post-cutoff libraries, released in autumn and winter 2021, record higher unadjusted download totals than libraries released in the preceding summer months.

Figure 5.1 makes this visible. The positive raw jump shows up at both the 26-week and 52-week horizons, consistent with a well-documented seasonal pattern in developer software adoption. The positive discontinuity in the Broad tier disappears entirely in the Successful subsample ($-0.195$, $p = 0.950$). That is the first signal of the barrier-to-entry mechanism.

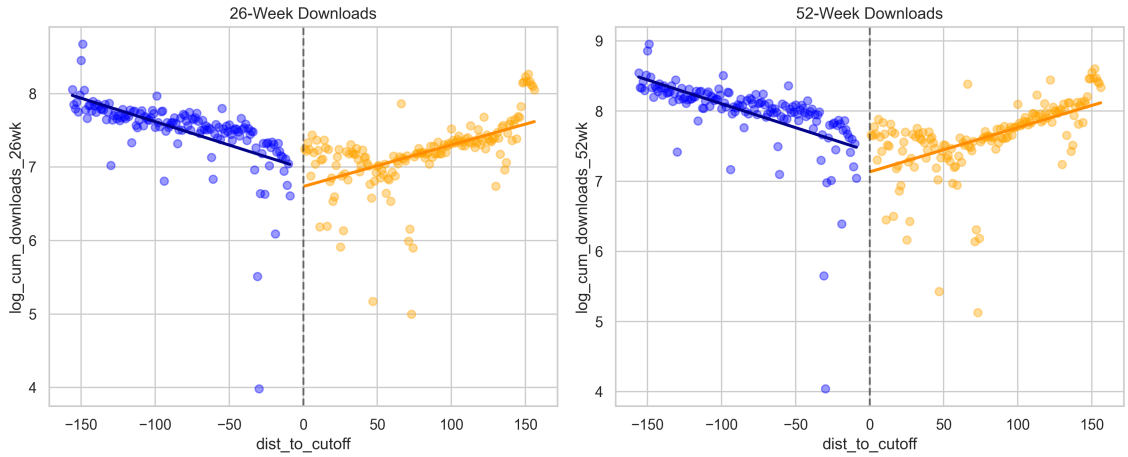


Figure 5.1 - Binned scatterplots of $\log(1+\text{downloads})$ by release week relative to the September 2021 cutoff, for 26-week (left) and 52-week (right) cumulative PyPI downloads. Each point represents a weekly bin average; fitted lines show local linear estimates on each side of the cutoff. The upward trend reflects the secular growth of the Python ecosystem; the positive discontinuity at the cutoff captures the raw seasonal jump, which the Diff-in-RDD design subsequently nets out. *Source:* Author's calculations based on PyPI data.

## 5.4 Difference-in-Discontinuities Estimates

The Diff-in-RDD stacks the 2021 cohort with three placebo cohorts (2018, 2019, 2020), estimates the RDD discontinuity for each year at the same calendar position, and extracts the 2021-specific excess. The excess jump is identified by the interaction term $\hat{\tau}^{2021} - \bar{\hat{\tau}}^{\text{placebo}}$, estimated via WLS.

In descriptive terms, libraries likely included in the training data based on their release timing average roughly three times more weekly PyPI downloads than narrowly excluded libraries in the post-ChatGPT period. That ratio grows to roughly 5:1 by early 2026, compared with a modest pre-ChatGPT baseline difference of about 28%.[1] The Diff-in-RDD formalizes the same pattern. The baseline-adjusted excess jump is $-7.3$ ($p = 0.002$) for the Successful tier and $-7.3$ ($p = 0.001$) for the Broad tier (Panel B of Table 5.3). The unadjusted estimates are larger in magnitude ($-12.8$ and $-9.1$, Panel C), which is expected. Adding $\log(1+\text{52-week downloads})$ as a covariate absorbs pre-existing level differences and roughly halves the standard errors, leaving a cleaner and more precise estimate of the ChatGPT-specific penalty.

The 52-week outcome provides a pre-ChatGPT baseline. For nearly all libraries within the $h = 13$ bandwidth, the 52-week accumulation window closes before November 2022, and libraries at dist $= +1$ (the nearest post-cutoff observation) have their 52-week window end in early October 2022, still eight weeks before ChatGPT. The 52-week Diff-in-RDD for the Successful tier ($-3.7$, $p = 0.026$, Panel A of Table 5.3) therefore reflects a pre-existing seasonal

[1] These descriptive ratios compare mean weekly downloads for libraries released in the four weeks before the donut against those released in the four weeks after the cutoff. The Diff-in-RDD estimates control for seasonal patterns via the placebo-year baseline.

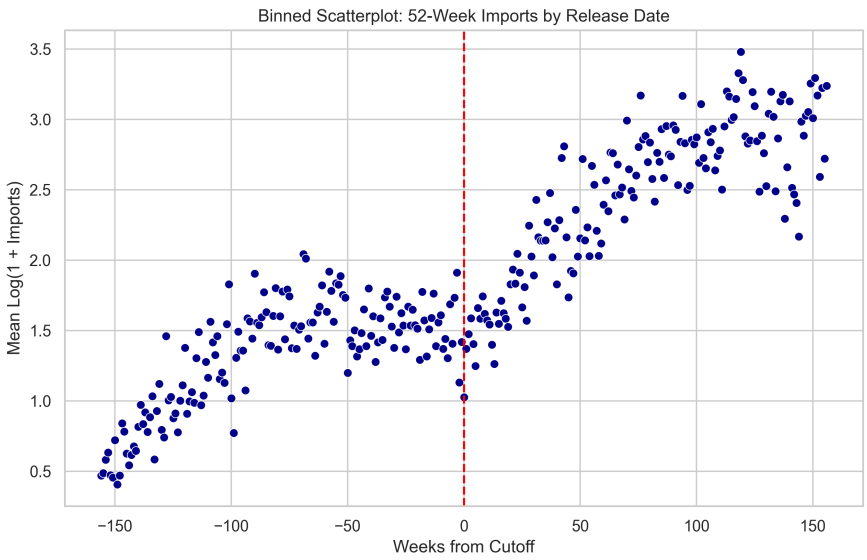


Figure 5.2 - Binned scatterplot of $\log(1 + 52\text{-week imports})$ by release week relative to the September 2021 cutoff (GitHub-matched subsample). Each point represents a weekly bin average. The upward trend reflects the secular growth of the Python ecosystem; the discontinuity at the cutoff captures the raw RDD effect, which mixes the Knowledge Wall penalty with the seasonal baseline. *Source:* Author's calculations based on GitHub data.

difference. The typical autumn advantage for post-September libraries was simply smaller in 2021 than in placebo years, even before LLMs became widely available. The ANCOVA specification in Panel B absorbs that pre-existing variation directly, so the separate netting calculation ($-12.8$ minus $-3.7$) is unnecessary. The adjusted estimate of $-7.3$ already reflects the ChatGPT-specific penalty net of baseline differences.

## 5.5 Pre-AI Covariate Balance

Before interpreting the post-AI suppression estimates, it is worth checking that libraries on either side of the cutoff were comparable in their *pre-AI* performance, measured before ChatGPT could have influenced adoption patterns. An RDD with a binary outcome, whether library $i$ crosses the 500-download threshold within its first 26 weeks, tests whether baseline human-developer adoption differs at the cutoff. For the 2021 cohort, the 26-week window closes around March 2022, entirely before ChatGPT.

Post-cutoff libraries are roughly 29 percentage points *more* likely to reach the Successful tier in their first 26 weeks than pre-cutoff libraries. This pre-AI advantage for post-cutoff libraries most likely reflects seasonal patterns in library release and early adoption. The important point is that it makes the post-AI suppression in Tables 5.3 and 5.5 *conservative*. Post-cutoff libraries start with a pre-AI advantage and still fall behind once LLMs become a mass interface. The Diff-in-RDD design, which nets out such seasonal baselines, remains the right estimator for the

Table 5.3 - Diff-in-RDD Estimates, 2021 Excess Jump

| Tier | Outcome | Excess Jump | Cluster SE | p-value | N |
|---|---|---|---|---|---|
| *Panel A: Pre-ChatGPT baseline (52-week downloads, unadjusted)* | | | | | |
| Successful (min 500) | 52-week Downloads | $-3.724$ | 1.669 | 0.026 | 75,313 |
| *Panel B: Post-ChatGPT activation (baseline-adjusted)* | | | | | |
| Broad (min 10) | Post-AI Downloads | $-7.285$ | 2.200 | 0.001 | 88,001 |
| Successful (min 500) | Post-AI Downloads | $-7.294$ | 2.393 | 0.002 | 75,313 |
| Superstar (min 1000) | Post-AI Downloads | $-0.251$ | 0.349 | 0.473 | 57,931 |
| *Panel C: Post-ChatGPT activation (unadjusted, for comparison)* | | | | | |
| Broad (min 10) | Post-AI Downloads | $-9.139$ | 4.425 | 0.039 | 88,001 |
| Successful (min 500) | Post-AI Downloads | $-12.792$ | 4.827 | 0.008 | 75,313 |
| Superstar (min 1000) | Post-AI Downloads | $-0.553$ | 0.492 | 0.261 | 57,931 |

*Notes:* WLS with year-by-week clustered standard errors. Triangular kernel, $h = 13$ weeks, 9-week asymmetric donut. Placebo cutoffs at equivalent calendar position in 2018, 2019, and 2020. All estimates are on the $\log(1+Y)$ scale. Panel B controls for $\log(1+\text{52-week downloads})$ as an ANCOVA-style baseline covariate; Panel A omits this covariate (circularity). For the Successful tier in the 2021 cohort, the sample mean (SD) of $\log(1+\text{post-AI downloads})$ is 8.98 (1.96). Because the excess jump is a difference-in-discontinuities (the 2021 cutoff jump minus the average placebo-year jump), it does not convert to a multiplicative effect via exponentiation; descriptive ratios are provided in the text for magnitude context.
*Source:* Author's calculations based on PyPI data.

Table 5.4 - Pre-AI Covariate Balance, Probability of Reaching the Successful Tier

| Outcome | Estimate (pp) | Robust SE | p-value | N |
|---|---|---|---|---|
| $\Pr(\text{downloads}_{26\text{wk}} \geq 500)$ | $+28.6$ | 3.94 | $< 0.001$ | 78,762 |

*Notes:* Linear probability model via `rdrobust`. Bias-corrected robust estimate. Triangular kernel, $h = 26$ weeks, 9-week asymmetric donut. Positive estimate indicates post-cutoff libraries are *more* likely to reach the threshold.
*Source:* Author's calculations based on PyPI data.

causal suppression effect.

## 5.6 The Implementation Gap - GitHub vs PyPI

A channel test of the LLM-mediated mechanism uses GitHub import counts. Table 5.5 reports single-year `rdrobust` estimates for the 2021 GitHub-matched subsample ($N = 28{,}243$).

These single-year estimates are large but fragile. They have no placebo baseline, making them sensitive to any pre-existing seasonal advantage at the September boundary. Bandwidth sensitivity (Appendix E) confirms this: at $h = 18$ the post-AI estimate falls from 26.8 to 2.9. To properly identify the GitHub channel, Table 5.6 applies the same Diff-in-RDD design used for PyPI to the GitHub-matched subsample, pooling 2018–2020 placebo cohorts with the 2021 main cohort.

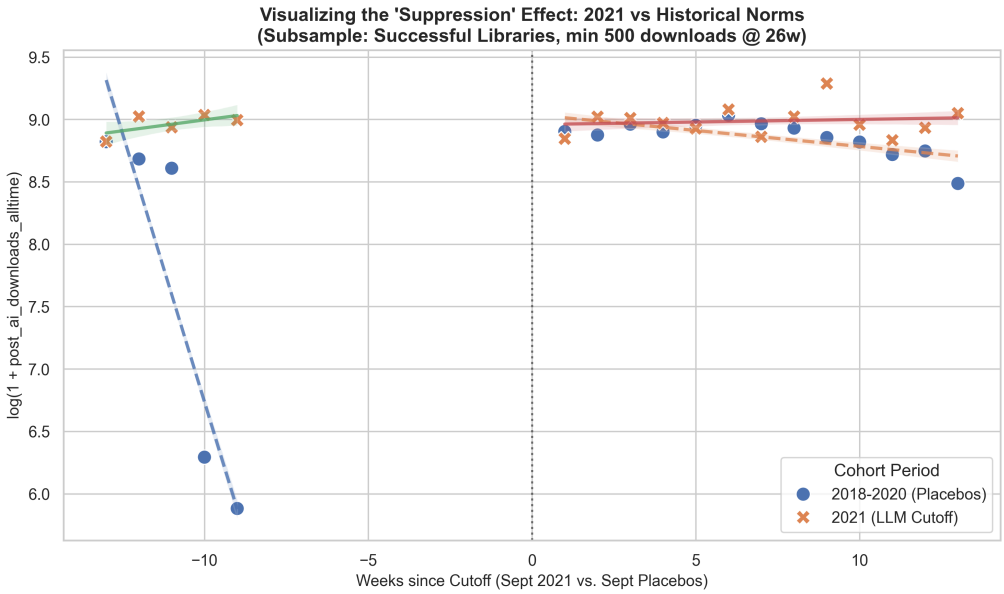


Figure 5.3 - Diff-in-RDD visualization for the Successful tier (min. 500 downloads at 26 weeks) - Blue circles: pooled 2018–2020 placebo cohorts. Orange crosses: 2021 cohort. Fitted lines show local linear estimates on each side of the cutoff. The 2021 cohort is shifted downward relative to the historical seasonal norm, identifying the Knowledge Wall penalty. *Source:* Author's calculations based on PyPI data.

The direction is consistent. All three GitHub import outcomes show a positive excess jump in 2021 relative to placebo years, with the all-time estimate reaching borderline significance ($p = 0.060$). On the same matched subsample, the PyPI Diff-in-RDD estimates are close to zero and insignificant. The contrast, positive GitHub effects alongside null PyPI effects on the same libraries, is consistent with LLM-steered code generation as the operative channel. However, the matched subsample contains only about 1,500 libraries per cohort year within the bandwidth, too few for the Diff-in-RDD to reach conventional significance. The implementation gap is therefore treated as suggestive directional evidence rather than a confirmed causal finding.

Figure 5.4 shows that the estimated single-year discontinuity grows monotonically from the 52-week horizon through the all-time horizon, in line with the activation hypothesis. The gap is dormant at release and only emerges once LLMs become a mass interface.

Figure 5.5 gives the key visual test of the activation hypothesis. The two cohorts differ modestly before November 2022, reflecting seasonal timing, after which the pre-cutoff advantage widens sharply and persistently. By early 2026 the ratio is approximately 5:1, with the gap still growing. The widening after ChatGPT, and its absence before, is descriptive evidence that the Knowledge Wall activates at mass LLM deployment rather than at the training cutoff itself.

The selection of the GitHub-matched subsample deserves attention, because it interacts with the interpretation of the implementation gap in a substantive way. Within the near-cutoff window of libraries released within 26 weeks of the September 2021 boundary (donut excluded), the GitHub-matched sample contains 1,804 pre-cutoff and 2,341 post-cutoff libraries. Among

Table 5.5 - Single-Year RDD Estimates, GitHub Implementation (2021 Only)

| Outcome | Estimate | SE | p-value | N |
|---|---|---|---|---|
| 52-week Imports | 11.554 | 10.778 | 0.284 | 28,243 |
| GPT-4 Era Imports | 23.256 | 11.541 | 0.044 | 28,243 |
| Post-AI Imports | 26.767 | 10.932 | 0.014 | 28,243 |
| All-Time Imports | 27.863 | 9.987 | 0.005 | 28,243 |

*Notes:* Bias-corrected robust estimates via `rdrobust`. Triangular kernel, $h = 13$ weeks, 9-week asymmetric donut. All estimates are on the $\log(1+Y)$ scale. These are single-year estimates without a placebo baseline.
*Source:* Author's calculations based on GitHub data.

Table 5.6 - Diff-in-RDD, GitHub versus PyPI on the Matched Subsample

| Channel | Outcome | Excess Jump | SE | p-value | N |
|---|---|---|---|---|---|
| GitHub | 52-week Imports | 1.194 | 1.931 | 0.536 | 6,582 |
| GitHub | Post-AI Imports | 1.887 | 1.157 | 0.103 | 6,582 |
| GitHub | All-Time Imports | 2.136 | 1.133 | 0.060 | 6,582 |
| PyPI | 52-week Downloads | 0.065 | 1.145 | 0.955 | 6,582 |
| PyPI | Post-AI Downloads | −2.244 | 3.901 | 0.565 | 6,582 |

*Notes:* WLS with triangular kernel, $h = 13$ weeks, 9-week asymmetric donut, year-by-week cluster-robust SEs. Broad tier (min. 10 downloads at 52 weeks). The GitHub-matched subsample contains approximately 1,500–2,100 libraries per cohort year within the estimation bandwidth. All outcomes are on the $\log(1+Y)$ scale.
*Source:* Author's calculations based on PyPI and GitHub data.

these, 86.8% of pre-cutoff libraries and 92.0% of post-cutoff libraries qualify as Successful (at least 500 pre-ChatGPT downloads), a 5.2 percentage point advantage for the post-cutoff group. The GitHub-matched post-cutoff sample is therefore more selected for demonstrated pre-AI quality than its pre-cutoff counterpart. This selection works against detecting a suppression effect, since post-cutoff libraries in the matched sample are disproportionately high-quality, and their post-AI import disadvantage cannot be attributed to a quality difference between the groups. The suppression estimate from this sample is accordingly conservative.

The selection pattern is also visible in implementation survival rates. Among GitHub-matched Successful-tier libraries in the near-cutoff window, 44.1% of pre-cutoff and 42.5% of post-cutoff libraries accumulate at least 10 post-AI all-time imports, and 14.3% of pre-cutoff versus 12.6% of post-cutoff libraries accumulate at least 100 imports. These raw differences are small in absolute terms and are not adjusted for running variable continuity. They are, however, directionally consistent with the RDD estimates. Post-cutoff libraries are less likely to achieve implementation traction at every threshold, despite being more selected for pre-AI quality. The RDD estimates (26.8 for post-AI imports, Table 5.5) pick up the sharp local discontinuity at the boundary that these sample-average rates smooth over.

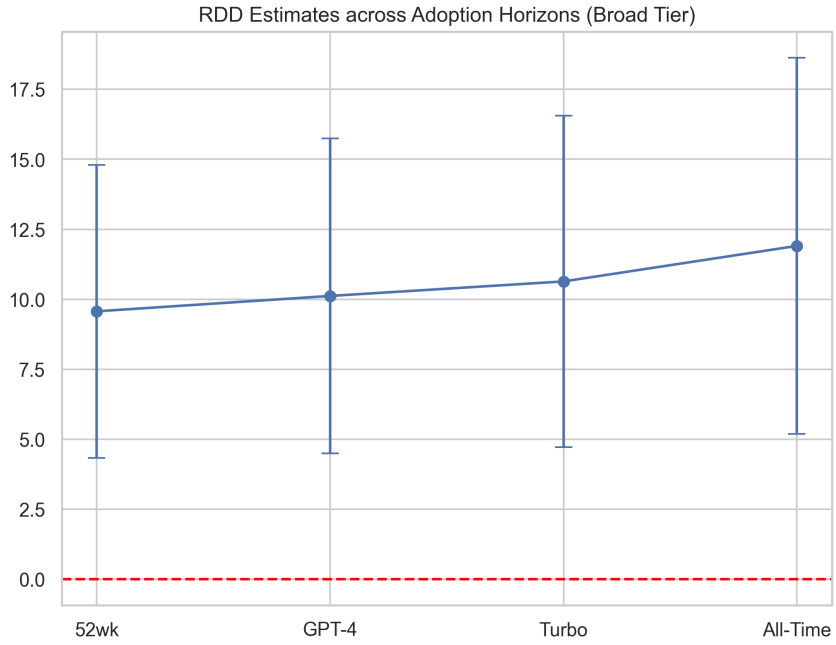


Figure 5.4 - RDD point estimates for PyPI downloads across adoption horizons (Broad tier). Vertical bars are 95% robust confidence intervals. The estimate grows from the 52-week horizon (mostly pre-ChatGPT) through the all-time horizon, consistent with the activation of the Knowledge Wall after November 2022. *Source:* Author's calculations based on PyPI data.

## 5.7 Mechanism Moderation Test

An exploratory test splits the sample by AI exposure intensity, above versus below the median of the commit-level AI score, and asks whether the Knowledge Wall penalty differs between the two groups. Neither interaction is statistically significant ($p = 0.179$ for 52-week downloads, $p = 0.505$ for baseline-adjusted post-AI downloads, $N = 1{,}303$). The test is almost certainly underpowered, with roughly 650 observations per group. The null cannot distinguish between a broad ecosystem-wide penalty and a targeted effect that the test simply lacks power to detect. Full specification and tables are reported in Appendix F.

## 5.8 Robustness Checks

The main Diff-in-RDD estimates are stable across bandwidths $h \in \{13, 15, 18, 26\}$ weeks in both sign and approximate magnitude (Appendix E). Replacing the asymmetric 9-week donut with a symmetric $\pm 4$-week exclusion produces qualitatively consistent results. The McCrary density test detects no significant discontinuity in the density of release dates at the cutoff (Figure 3.1), so there is no evidence of sorting around the boundary. A moving-cutoff permutation test confirms that the raw RDD estimate is not unusual relative to arbitrary placebo dates ($p = 0.344$), which is itself part of the case for the Diff-in-RDD design (Appendix B). The GitHub implementation estimates are more sensitive to bandwidth choice than the PyPI

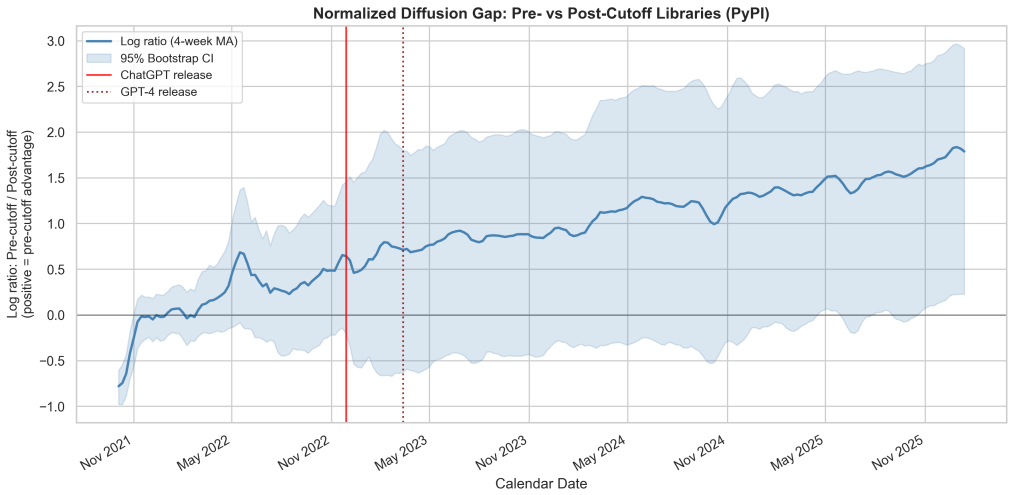


Figure 5.5 - Normalized diffusion gap between pre- and post-cutoff libraries (PyPI) - The $y$-axis plots the log ratio of mean weekly downloads for the July 2021 (pre-cutoff) cohort relative to the October 2021 (post-cutoff) cohort; values above zero indicate a pre-cutoff advantage. The shaded band shows 95% bootstrap confidence intervals (500 replications, resampled at the library level). Before ChatGPT (November 2022), the ratio fluctuates around a modest baseline of roughly 28%. After ChatGPT, the gap widens steadily and reaches approximately 5:1 by early 2026, with no evidence of closing. *Source:* Author's calculations based on PyPI data.

Diff-in-RDD, and I therefore treat them as descriptive evidence consistent with the channel interpretation rather than as independently robustified causal estimates.

# Chapter 6 – Discussion

## 6.1 What the Evidence Shows

The question is whether Python libraries released just before the GPT-3.5/GPT-4 training cutoff diffuse more widely in the AI era than those released just after, because the former are more likely to be represented in the model's pretrained knowledge. The evidence here points to yes, though with important caveats about the strength of causal inference in observational settings. The main finding rests on solid ground as a quasi-experimental estimate; the mechanism evidence is weaker.

A gap in weekly downloads is the clearest way to state what happened. Among libraries that had demonstrated pre-ChatGPT quality, those released just before September 2021 now average roughly three times more weekly PyPI downloads than those released just after, and the ratio grows to approximately five times by early 2026. Before ChatGPT, the same two groups differed by only about 28%, roughly the autumn release premium. The baseline-adjusted Diff-in-RDD then confirms that neither seasonal timing nor pre-existing quality can explain the post-ChatGPT gap. Its 2021-specific excess discontinuity in log post-AI cumulative downloads is $-7.3$ ($p = 0.002$), significant at the 1% level after netting out 2018–2020 placebo seasonality and controlling for each library's pre-ChatGPT scale.[1] The finding survives placebo testing and is stable across bandwidths. A properly identified Diff-in-RDD on the GitHub-matched subsample finds directionally larger effects for code imports than for downloads (Table 5.6), consistent with LLM-generated code as the channel, though the matched sample is too small for conventional significance. No comparable discontinuity appears in the 2018–2020 placebo cohorts, and the gap only opens after November 2022; together, the two patterns match the proposed mechanism. The RDD design provides strong quasi-experimental identification that approaches, but does not fully achieve, the conditions of a randomized experiment; the causal interpretation accordingly rests on the plausibility of the identifying assumptions rather than on random assignment.

## 6.2 The Knowledge Wall

The findings are organized around the concept of a *Knowledge Wall*: a structural discontinuity in the information landscape of deployed LLMs, arising from the hard temporal boundary of their training data. Libraries on the pre-cutoff side of the wall are represented in the model's parametric knowledge and are therefore candidates for recommendation, code generation, and

[1] This log-unit figure is a difference of four RDD jumps (2021 minus the 2018–2020 placebo average), not a single log-ratio, so it does not convert to a simple percentage; the descriptive ratios above are the interpretable magnitude.

documentation synthesis. Libraries on the post-cutoff side are invisible to this channel.

This framing has three implications:

1. The Knowledge Wall is an architectural property of static-knowledge models, present regardless of developer awareness.

2. The wall is asymmetric; pre-cutoff libraries benefit from the absence of a barrier that post-cutoff libraries face.

3. The effects are expected to be persistent. As of early 2026, no catch-up is observed in PyPI downloads (Figure 6.2), and the GitHub Diff-in-RDD shows a directionally growing gap across horizons (Table 5.6), suggesting the divergence may have accumulated rather than narrowed.

The absence of a significant effect for the Superstar tier (at least 1,000 pre-ChatGPT downloads, $p = 0.473$, baseline-adjusted) is consistent with this framing. Libraries that achieved the highest early visibility before ChatGPT are more likely to have accumulated organic adoption through channels largely independent of LLM recommendation: conference presentations, influential blog posts, direct dependency chains, and word-of-mouth within specialist communities. The Knowledge Wall appears most consequential at the margin of entry, where LLM-mediated discovery is most relevant to adoption; it does not appear to override momentum already built through high-visibility human adoption. Figure 6.1 summarizes this gradient across the three success tiers.

## 6.3 Connection to Doshi and Hauser (2024)

The conceptual foundation draws on Doshi and Hauser (2024), who show that generative AI improves individual creative quality while reducing collective diversity through convergence. The parallel here is structural. AI steers developers toward convergent tool choices, specifically toward tools that were established before the knowledge cutoff. Individual developers gain productivity, the individual-enhancement side, but the aggregate effect is a concentration of adoption around the pre-cutoff library set, the collective-narrowing side.

There is, however, an important structural difference. In Doshi and Hauser (2024), the narrowing arises because all participants converge toward similar high-quality outputs: the model nudges users toward the same solutions, and diversity compresses from above. In the software ecosystem, the narrowing arises through informational exclusion: post-cutoff libraries are not recommended at all, regardless of their quality, and pre-cutoff libraries receive a relative advantage simply by being visible to the intermediary. The mechanism is absence rather than convergence. The distinction matters because it implies a different kind of policy response. Convergence effects might be mitigated by diversifying model outputs, while exclusion effects require updating the model's knowledge base.

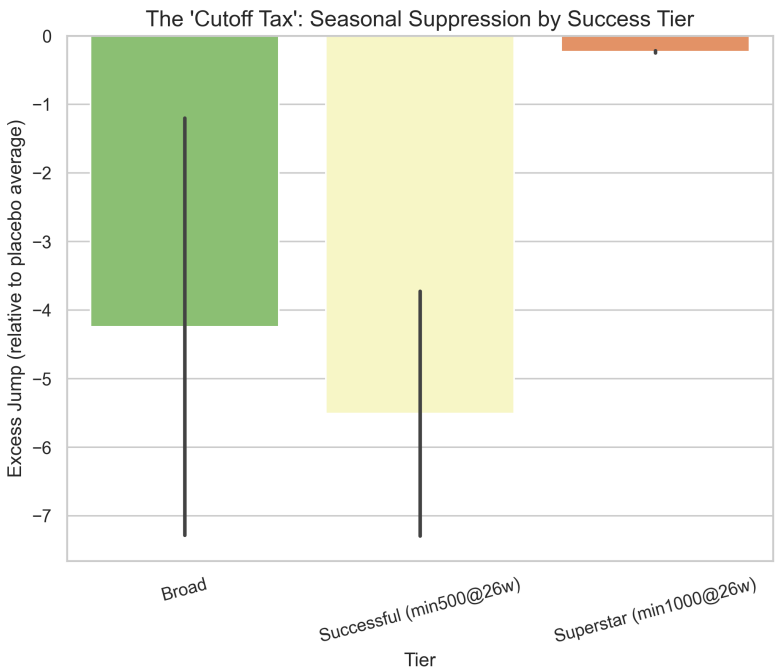


Figure 6.1 - Diff-in-RDD excess jump (relative to 2018–2020 placebo average) by success tier. All estimates on the $\log(1+Y)$ scale. The suppression is concentrated in the Broad and Successful tiers; the Superstar tier shows no significant penalty, consistent with high-visibility libraries maintaining adoption through channels independent of LLM recommendation. *Source:* Author's calculations based on PyPI data.

This reading is consistent with the large-scale observational evidence in Daniotti et al. (2025), who find that AI use is associated with increased exploration of new libraries in aggregate. The two findings are not in tension. AI assistance expands the total set of libraries developers encounter and try, and the Knowledge Wall may determine which subset of that expanded exploration the model can actively recommend and implement. The net result is an asymmetric boost. Pre-cutoff libraries benefit from the AI-mediated expansion in library usage, while post-cutoff libraries are excluded from the channel through which that expansion propagates. The Knowledge Wall is therefore best understood not as a suppressor of aggregate experimentation but as a distributional filter on its direction.

## 6.4 Alternative Mechanisms

The most immediate alternative explanation for a discontinuity at September 2021 is *calendar seasonality*, with post-summer releases entering a high-activity development period. That is what the Diff-in-RDD is designed to remove. The placebo cohorts (2018–2020) supply an estimate of the typical September discontinuity, and the 2021 excess is what remains after that baseline is subtracted. The significance of the baseline-adjusted excess jump ($p = 0.002$ for the Successful tier) is therefore not attributable to seasonal patterns.

A more subtle threat is *quality sorting*, where libraries on either side of the cutoff differ

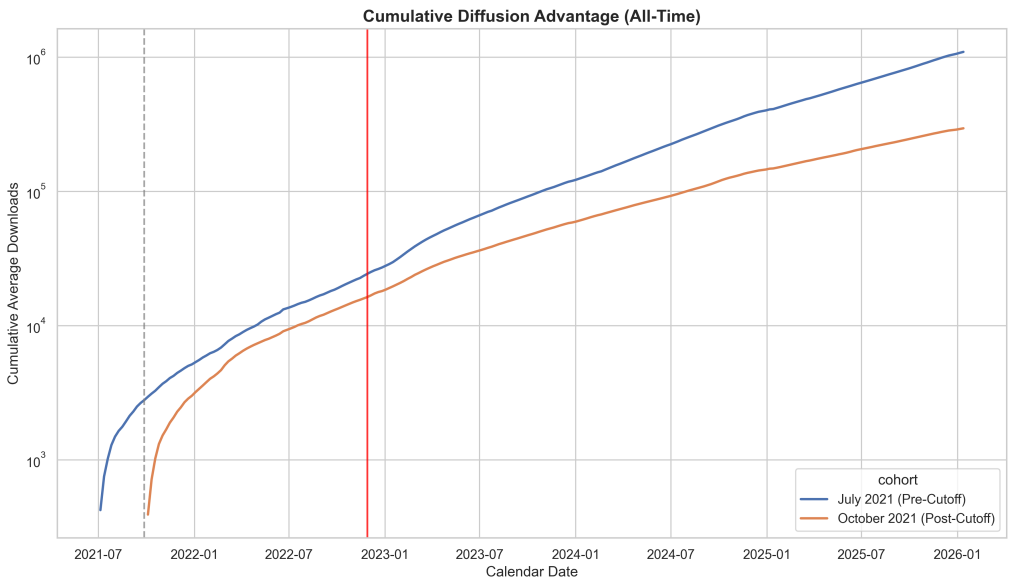


Figure 6.2 - Cumulative average PyPI downloads for the July 2021 (pre-cutoff) and October 2021 (post-cutoff) cohorts. The vertical red line marks ChatGPT release (November 2022). The gap between cohorts widens monotonically after November 2022, with no catch-up through January 2026. By early 2026, pre-cutoff libraries have accumulated roughly five times the weekly download volume of post-cutoff libraries. *Source:* Author's calculations based on PyPI data.

systematically in quality. The success tier logic handles this directly. Because the treatment here is the arrival of ChatGPT in November 2022, conditioning on pre-ChatGPT performance (at least 500 downloads in the first 26 weeks) is not selection on the dependent variable. It simply keeps libraries that human developers had already judged comparable before LLMs could have influenced adoption patterns. The excess jump persists in this subsample, so quality sorting is an unlikely explanation.

Additional threats include the exact placement of the cutoff within September 2021 (handled by the 9-week donut), the concurrent deployment of GitHub Copilot (which shares the same approximate knowledge boundary, so the finding is a property of the model ecosystem rather than of a single product), general platform dynamics (addressed by the GitHub Diff-in-RDD channel test), and retrieval-augmented generation capabilities (which should erode the gap if active, yet Figure 5.5 shows it widening through 2026). These threats are discussed in detail in Appendix C, and none alters the main conclusion.

## 6.5 The Systemic Wall Interpretation

The mechanism moderation test finds no significant heterogeneity between High and Low AI exposure libraries ($p = 0.179$ for 52-week downloads, $p = 0.505$ for baseline-adjusted post-AI downloads). This is consistent with the Knowledge Wall functioning as a broad ecosystem-level phenomenon rather than a targeted penalty on identifiable AI-mediated usage. LLMs function

as a general discovery and documentation interface, and exclusion from that interface reduces discoverability across the whole software stack, not only in AI-heavy contexts.

If the suppression were concentrated among libraries with high AI exposure scores, the implication would be that only the code generation channel matters and that libraries adopted primarily through human channels are unaffected. The null result suggests a different interpretation: the Knowledge Wall reshapes the information environment in which all adoption decisions are made. Developers who do not use AI coding assistants directly still encounter the consequences of the wall indirectly, through reduced presence in tutorials, Stack Overflow answers, blog posts, and dependency chains that are themselves shaped by LLM-generated content. The boundary between AI-mediated and human-mediated discovery is porous in practice. This reading is consistent with the theoretical prediction of Kleinberg and Raghavan (2021) that correlated algorithmic inputs produce correlated population-level outcomes even when individual decision-makers are not themselves algorithmic.

## 6.6 Scope Conditions and External Validity

The estimate is a Local Average Treatment Effect at the September 2021 boundary, identified from a single cutoff in the Python ecosystem. It should not be extrapolated to libraries released far from the cutoff, to other cutoff dates, or to programming languages with less AI-mediated developer activity without further evidence. Python is a particularly susceptible ecosystem because it is the dominant language for data science and machine learning, the domains in which AI-assisted coding is most prevalent (Stack Overflow, 2024; Daniotti et al., 2025). The effect size identified here may be an upper bound relative to ecosystems with less AI-mediated developer activity, such as systems programming languages where developers rely more on compiler toolchains and less on LLM-generated code. Replication in npm (JavaScript), crates.io (Rust), and CRAN (R) would establish whether the Knowledge Wall is a general feature of LLM-mediated software adoption or one specific to Python's AI-adjacent community. Detailed scope conditions are discussed in Appendix D.

## 6.7 Limitations

Key limitations include running variable measurement error (PyPI registration lags bias estimates conservatively), matching coverage (the 5.3% GitHub match rate over-represents prominent libraries), the endogeneity of the AI exposure moderator, and the absence of family-wise error correction across multiple outcomes and tiers. The baseline-adjusted Diff-in-RDD for the Successful tier ($p = 0.002$) is designated as the main specification *ex ante*, and the Broad-tier estimate ($p = 0.001$) is comparably significant after baseline adjustment. The causal claim rests on the parallel trends assumption, which is supported by placebo stability but remains

fundamentally untestable. Detailed discussion of multiple testing and secular growth threats is provided in Appendix D.

## 6.8 Implications

The findings have meaningful implications for multiple groups. For library authors, release timing relative to major LLM cutoffs appears to be a predictor of adoption probability, suggesting that maintainers of newer libraries may need to invest disproportionately in alternative discovery channels to compensate for the Knowledge Wall. For the software ecosystem, if LLMs function as concentration mechanisms, successive model generations may reinforce existing adoption hierarchies and potentially slow the turnover of the dominant tool set. Finally, for model providers, the results imply that knowledge freshness is a first-order design parameter, and mitigations such as more frequent retraining or improved retrieval-augmented generation may be necessary to ensure the AI-mediated discovery channel remains open to new innovation.

# Chapter 7 – Conclusion

This paper asked whether the training-data cutoff of large language models affects the diffusion of newly released open-source software. Exploiting the September 2021 knowledge boundary shared by the GPT-3.5 and GPT-4 model families, I estimated a regression discontinuity in the adoption trajectories of Python libraries released just before and just after the cutoff, using PyPI download counts and GitHub import data. A Difference-in-Discontinuities design, which differences the 2021 discontinuity against placebo discontinuities at the same calendar position in 2018–2020, provides the primary causal estimate.

The answer is yes. Libraries released just after the cutoff, and therefore excluded from the dominant LLMs' parametric knowledge, accumulate significantly fewer downloads and code imports once those models reach mass deployment. Before ChatGPT, the two groups differed by roughly 28% in weekly downloads, a modest seasonal baseline. By early 2026, pre-cutoff libraries average approximately five times more weekly downloads, and the gap continues to widen. The baseline-adjusted Diff-in-RDD estimate for libraries that had demonstrated pre-AI quality (at least 500 downloads within 26 weeks, measured entirely before ChatGPT) is $-7.3$ ($p = 0.002$). A Diff-in-RDD on the GitHub-matched subsample finds directionally larger effects for code imports than for downloads ($p = 0.060$ for all-time imports), consistent with LLM-steered code generation as the operative channel, though the matched sample is too small for conventional significance. Three placebo cohorts show no comparable discontinuity, and the gap only activates after November 2022. The suppression is concentrated in the Broad and Successful tiers, while libraries with the highest pre-ChatGPT visibility (Superstar tier, at least 1,000 downloads) show no significant penalty ($p = 0.473$), consistent with very high-visibility libraries holding their adoption through channels independent of LLM recommendation.

These findings support the existence of what this paper calls a Knowledge Wall, a structural barrier arising from the hard temporal boundary of LLM training data. The wall is associated with a redirection of early adoption toward libraries already represented in the model's knowledge, disadvantaging post-cutoff entrants regardless of their demonstrated quality. The mechanism is consistent with the broader theoretical concern, articulated by Doshi and Hauser (2024) in the context of creative output, that generative AI may enhance individual capability while narrowing collective variety. In the software ecosystem, the variety that narrows is the set of tools in active use, the individual enhancement is the productivity gain from AI-assisted coding, and the structural source of this asymmetry is the fixed knowledge boundary that shapes which tools the AI can recommend.

This paper makes three contributions. First, it provides a quasi-experimental estimate of the causal effect of an LLM training boundary on software adoption, exploiting a precisely dated institutional feature for identification. Second, it introduces a two-outcome design, PyPI downloads versus GitHub imports, that provides suggestive evidence for the code generation

channel as distinct from general discoverability. Third, it shows that a pre-AI quality advantage for post-cutoff libraries (28.6 percentage points more likely to reach the Successful tier before ChatGPT) reverses into a post-AI deficit, which establishes that the suppression cannot be attributed to pre-existing differences in library quality.

Several limitations bound the conclusions. The running variable is constructed from first observed download dates, a noisy proxy for true release timing that biases estimates conservatively. The training cutoff date itself is documented only at the month level. The specific day (September 27) is a researcher assignment, and the asymmetric donut exclusion is the primary defense against the ambiguity. The GitHub-matched subsample covers 5.3% of the PyPI universe and over-represents prominent libraries, which limits both the external validity and the statistical power of the implementation gap estimates; the GitHub Diff-in-RDD is directionally consistent but does not reach conventional significance ($p = 0.060$ for all-time imports). The exploratory mechanism test, asking whether suppression is stronger among libraries with higher AI-exposure scores, is non-significant ($p = 0.179$ for 52-week downloads, $p = 0.505$ for baseline-adjusted post-AI downloads), but the moderator is post-treatment and the test is underpowered ($N = 1{,}303$), so the null cannot distinguish a broad ecosystem effect from an undetectable targeted one. Finally, the estimates are local to a narrow window around a single cutoff date and should not be extrapolated to the full ecosystem or to distant time horizons without further evidence.

Several questions remain open. As model providers adopt more frequent retraining schedules and retrieval-augmented generation becomes standard, the knowledge asymmetry may diminish. Whether the Knowledge Wall is a transitional feature of the current generation of static-knowledge models, or a durable property of AI-mediated ecosystems, is an empirical question that can only be answered longitudinally. Replication at other cutoff dates, for Claude, Gemini, Llama, and subsequent GPT versions, would establish whether the finding generalizes beyond the September 2021 boundary. Replication in other language ecosystems (npm, crates.io, CRAN) would test whether the effect is specific to Python's AI-adjacent developer community or a general feature of LLM-mediated software adoption. The mechanism test deserves a dedicated design as well, since a larger matched sample with an exogenous measure of AI exposure could determine whether the wall operates through code generation specifically or through a broader shift in developer information acquisition. Finally, the welfare implications of the Knowledge Wall, whether the redirection of adoption toward pre-cutoff tools is costly or merely redistributive, depend on the substitutability of pre- and post-cutoff libraries, a question this paper does not address.

The broader implication is that LLMs, by fixing a temporal boundary in their training data and mediating an increasing share of developer decision-making, have introduced what appears to be a new structural factor in technology adoption. The Knowledge Wall is not the result of deliberate gatekeeping or market power. It is an architectural byproduct of how these models are built. Its consequences, however, are distributional, and they fall on the developers and

projects that happen to arrive after the boundary. Understanding and mitigating that asymmetry is a design problem for the next generation of AI-assisted development tools.

# Bibliography


Acemoglu, D. (2024). The simple macroeconomics of AI. Working Paper 32487, National Bureau of Economic Research.

Anderson, B. R., Shah, J. H., and Kreminski, M. (2024). Homogenization effects of large language models on human creative ideation. In *Proceedings of the 16th Conference on Creativity & Cognition*, pages 413–425. ACM.

Arthur, W. B. (2009). *The Nature of Technology: What It Is and How It Evolves*. Free Press, New York.

Azoulay, P., Fons-Rosen, C., and Graff Zivin, J. S. (2019). Does science advance one funeral at a time? *American Economic Review*, 109(8):2889–2920.

Barreca, A. I., Lindo, J. M., and Waddell, G. R. (2016). Heaping-induced bias in regression-discontinuity designs. *Economic Inquiry*, 54(1):268–293.

Bass, F. M. (1969). A new product growth for model consumer durables. *Management Science*, 15(5):215–227.

Bommasani, R., Creel, K. A., Kumar, A., Jurafsky, D., and Liang, P. (2022a). Picking on the same person: Does algorithmic monoculture lead to outcome homogenization? In *Advances in Neural Information Processing Systems*, volume 35.

Bommasani, R., Hudson, D. A., Adeli, E., et al. (2022b). On the opportunities and risks of foundation models. Technical report, Stanford Institute for Human-Centered AI.

Borges, H., Hora, A., and Valente, M. T. (2016). Understanding the factors that impact the popularity of GitHub repositories. In *Proceedings of the IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 334–344.

Borges, H. and Valente, M. T. (2018). What's in a GitHub star? Understanding repository starring practices in a social coding platform. *Journal of Systems and Software*, 146:112–129.

Bresnahan, T. F. and Trajtenberg, M. (1995). General purpose technologies: Engines of growth? *Journal of Econometrics*, 65(1):83–108.

Brynjolfsson, E., Li, D., and Raymond, L. R. (2023). Generative AI at work. Working Paper 31161, National Bureau of Economic Research.

Calonico, S., Cattaneo, M. D., and Farrell, M. H. (2022). Coverage error optimal confidence intervals for local polynomial regression. *Bernoulli*, 28(4):2998–3022.

Calonico, S., Cattaneo, M. D., Farrell, M. H., and Titiunik, R. (2019). Regression discontinuity designs using covariates. *The Review of Economics and Statistics*, 101(3):442–451.

Calonico, S., Cattaneo, M. D., Farrell, M. H., and Titiunik, R. (2020). Optimal bandwidth choice for robust bias-corrected inference in regression discontinuity designs. *The Econometrics Journal*, 23(2):192–210.

Calonico, S., Cattaneo, M. D., and Titiunik, R. (2014). Robust nonparametric confidence intervals for regression-discontinuity designs. *Econometrica*, 82(6):2295–2326.

Cattaneo, M. D., Idrobo, N., and Titiunik, R. (2020a). A practical introduction to regression discontinuity designs: Foundations. *Cambridge Elements: Quantitative and Computational Methods for Social Science*.

Cattaneo, M. D., Jansson, M., and Ma, X. (2020b). Simple local polynomial density estimators. *Journal of the American Statistical Association*, 115(531):1449–1455.

Cattaneo, M. D. and Titiunik, R. (2022). Regression discontinuity designs. *Annual Review of Economics*, 14:821–851.

Cheng, J., Marone, M., Weller, O., Lawrie, D., Khashabi, D., and Van Durme, B. (2024). Dated data: Tracing knowledge cutoffs in large language models. In *Conference on Language Modeling*. Outstanding Paper Award.

Daniotti, S., Wachs, J., Feng, X., and Neffke, F. (2025). Who is using AI to code? Global diffusion and impact of generative AI. arXiv preprint.

Dell, M. (2010). The persistent effects of Peru's mining mita. *Econometrica*, 78(6):1863–1903.

Doshi, A. R. and Hauser, O. P. (2024). Generative AI enhances individual creativity but reduces the collective diversity of novel content. *Science Advances*, 10(28):eadn5290.

Eloundou, T., Manning, S., Mishkin, P., and Rock, D. (2024). GPTs are GPTs: Labor market impact potential of LLMs. *Science*, 384(6702):1306–1308.

Fan, J. and Gijbels, I. (1996). *Local Polynomial Modelling and Its Applications*, volume 66 of *Monographs on Statistics and Applied Probability*. Chapman & Hall/CRC.

Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., and Wang, H. (2024). Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*.

Gelman, A. and Imbens, G. (2019). Why high-order polynomials should not be used in regression discontinuity designs. *Journal of Business & Economic Statistics*, 37(3):447–456.

Grembi, V., Nannicini, T., and Troiano, U. (2016). Do fiscal rules matter? *American Economic Journal: Applied Economics*, 8(3):1–30.

Hahn, J., Todd, P., and Van der Klaauw, W. (2001). Identification and estimation of treatment effects with a regression-discontinuity design. *Econometrica*, 69(1):201–209.

Hausman, C. and Rapson, D. S. (2018). Regression discontinuity in time: Considerations for empirical applications. *Annual Review of Resource Economics*, 10:533–552.

Imbens, G. and Kalyanaraman, K. (2012). Optimal bandwidth choice for the regression discontinuity estimator. *The Review of Economic Studies*, 79(3):933–959.

Imbens, G. W. and Lemieux, T. (2008). Regression discontinuity designs: A guide to practice. *Journal of Econometrics*, 142(2):615–635.

Kleinberg, J. and Raghavan, M. (2021). Algorithmic monoculture and social welfare. *Proceedings of the National Academy of Sciences*, 118(22):e2018340118.

Lee, D. S. and Lemieux, T. (2010). Regression discontinuity designs in economics. *Journal of Economic Literature*, 48(2):281–355.

McCrary, J. (2008). Manipulation of the running variable in the regression discontinuity design: A density test. *Journal of Econometrics*, 142(2):698–714.

Noy, S. and Zhang, W. (2023). Experimental evidence on the productivity effects of generative artificial intelligence. *Science*, 381(6654):187–192.

OpenAI (2022). Introducing ChatGPT. `https://openai.com/blog/chatgpt`. November 30, 2022.

OpenAI (2023). GPT-4 technical report. Technical report, OpenAI.

Peng, S., Kalliamvakou, E., Cihon, P., and Demirer, M. (2023). The impact of AI on developer productivity: Evidence from GitHub Copilot. *arXiv preprint arXiv:2302.06590*.

Pezik, P. et al. (2025). LLMLagBench: Identifying temporal training boundaries in large language models. *arXiv preprint arXiv:2511.12116*.

Python Software Foundation (2024). PyPI download statistics via BigQuery. `https://packaging.python.org/en/latest/guides/analyzing-pypi-package-downloads/`. Accessed: 2025.

Raghavan, M. (2024). Competition and diversity in generative AI. *arXiv preprint arXiv:2412.08610*.

Rogers, E. M. (2003). *Diffusion of Innovations*. Free Press, New York, 5th edition.

Shan, S., Ding, W., Passananti, J., Zheng, H., and Zhao, B. Y. (2024). Nightshade: Prompt-specific poisoning attacks on text-to-image generative models. In *Proceedings of the 45th IEEE Symposium on Security and Privacy*.

Shinoki, S. T., Picchetti, P., and Pinto, C. C. d. X. (2024). Difference-in-discontinuities: Estimation, inference and validity tests. *arXiv preprint arXiv:2405.18531*.

Stack Overflow (2024). Stack overflow developer survey 2024. `https://survey.stackoverflow.co/2024/`. Accessed: 2025.

Weitzman, M. L. (1998). Recombinant growth. *The Quarterly Journal of Economics*, 113(2):331–360.

Wu, F., Black, E., and Chandrasekaran, V. (2025). Generative monoculture in large language models. In *International Conference on Learning Representations*.

# Chapter A – Declaration of AI Tool Usage

In accordance with the BCE TDK competition rules (Section IV.1.b), the following declaration describes all uses of artificial intelligence tools during the preparation of this paper. Three AI tools were used, each restricted to minor subtasks as defined by the competition rules. *Claude* (Anthropic, Claude Code CLI) was used for literature search and orientation (identifying potentially relevant papers for manual verification), language and grammar checking of English-language prose, and LaTeX formatting assistance (table layout, equation typesetting, document structure). *Gemini CLI* (Google) was used for brainstorming research design alternatives, exploring the structure of robustness checks, and debugging Python scripts during data pipeline development. *GitHub Copilot* (Microsoft) was used for code autocompletion during the development of data processing and estimation scripts in Python.

During the research process, Claude and Gemini were used interactively to explore ideas, outline arguments, and produce preliminary drafts that served as working material. The final text was written by the author. All empirical results were produced by the author's own code executing established statistical packages (`rdrobust`, `statsmodels`, `pandas`). The research question, identification strategy, variable definitions, specification choices, and interpretation of findings are the author's own work, developed in consultation with the supervisor. No AI tool autonomously generated empirical results, made research design decisions, or produced text that appears in the submitted version without having been rewritten by the author.

# Chapter B – Permutation Inference

A non-parametric permutation test offers a distribution-free assessment of whether the discontinuity at the September 2021 cutoff is unusual relative to what would be observed at arbitrary dates. Following the moving-cutoff logic common in the causal inference literature (e.g., Dell, 2010), the primary RDD model is re-estimated at 157 alternative placebo Mondays between January 2019 and January 2022.

For each iteration, a hypothetical treatment is defined relative to a pseudo-cutoff date ($c'$), the running variable is recomputed, and the local linear discontinuity is estimated for the primary outcome, cumulative 52-week downloads. The procedure yields an empirical null distribution of RDD coefficients. The empirical $p$-value is

$$p_{perm} = \frac{1}{N}\sum_{i=1}^{N} \mathbb{I}(|\hat{\beta}_{placebo,i}| \geq |\hat{\beta}_{true}|).$$

The two-sided empirical $p$-value is 0.344 for the raw RDD estimate at the September 2021 cutoff. The raw 52-week download discontinuity at the true cutoff is therefore not unusual compared with discontinuities observed at arbitrary placebo dates. Figure B.1 shows the distribution of placebo RDD coefficients with the true estimate marked.

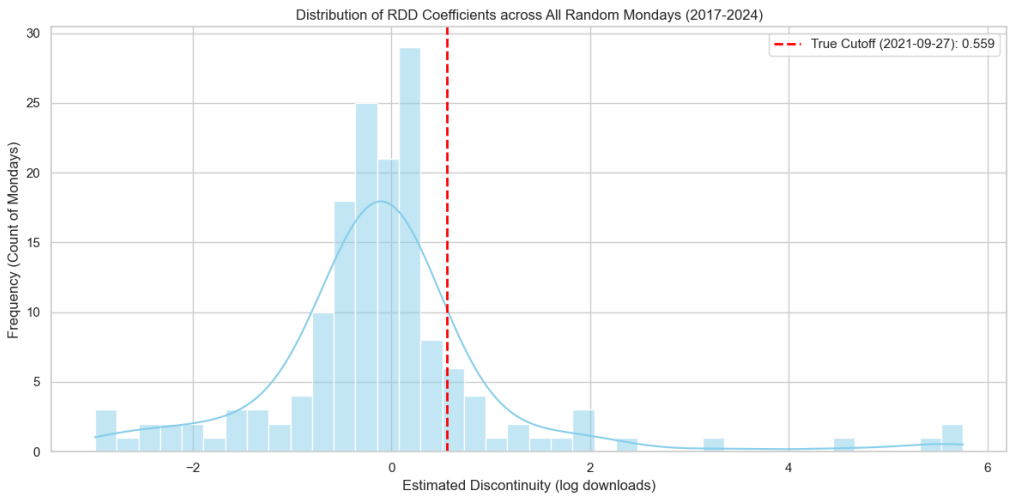


Figure B.1 - Distribution of RDD coefficients across 157 placebo Monday cutoff dates (2017–2024). The dashed red line marks the true September 2021 cutoff estimate (+0.559). The true estimate falls within the body of the placebo distribution ($p = 0.344$), confirming that the raw RDD is not informative in isolation. *Source:* Author's calculations based on PyPI data.

This result is expected. The raw RDD captures a composite of the Knowledge Wall effect and a seasonal pattern common to all September release windows. The permutation test cannot isolate the cutoff-specific component, which is exactly why the Diff-in-RDD, not the raw RDD, serves as the primary causal specification.

# Chapter C – Alternative Mechanisms (Extended)

This appendix discusses threats to interpretation that are summarized in the main text (Section 6.4).

## Exact Cutoff Location

OpenAI's documentation states that the GPT-3.5 and GPT-4 training data ends in "September 2021" without specifying a day. The cutoff date used in this paper, September 27 (the last Monday of the month), is a researcher choice rather than an institutionally documented boundary. The true cutoff could be September 1, September 15, or any other date in the month. This ambiguity is the main motivation for the 9-week asymmetric donut, which drops all libraries released in August and September 2021 from the estimation sample. Libraries in that window have genuinely uncertain training inclusion status, and removing them ensures the comparison is between libraries clearly before the cutoff (July and earlier) and libraries clearly after it (October and later). The estimates are therefore robust to any cutoff placement within September, at the cost of a smaller effective sample near the boundary. More broadly, the running variable, weeks relative to a date that is not precisely documented, has non-classical measurement error. In the RDD context, measurement error in the running variable biases estimates toward zero, which makes the reported effects conservative.

## GitHub Copilot

Copilot was released in public preview in June 2021, three months before the September 2021 cutoff, and its underlying Codex model was trained on a corpus with a similar temporal boundary. Because Copilot and the GPT-3.5/GPT-4 family share an approximately overlapping knowledge cutoff, the estimated suppression cannot be attributed to a single model. The September 2021 boundary instead represents a shared knowledge cutoff across the dominant LLM-based coding tools deployed in 2022–2023. The causal claim is therefore best read as the effect of the *knowledge boundary as a class*, that is, the date at which the major deployed LLMs stopped ingesting new training data, rather than the effect of any single product. This does not weaken the finding. It implies that the Knowledge Wall is a property of the model ecosystem's shared training infrastructure, not an idiosyncratic feature of ChatGPT alone.

## General Platform Dynamics

Any technology platform that directs developer attention could in principle produce comparable discontinuities. If the suppression were driven by a general attention platform, there would be no reason to expect GitHub code imports to show a larger excess jump than PyPI downloads on the same matched subsample. The GitHub Diff-in-RDD finds a positive excess jump for all three import horizons (the all-time estimate reaches $p = 0.060$) while the PyPI Diff-in-RDD on the same libraries is essentially zero. This directional contrast is consistent with the LLM code generation channel specifically, though the matched subsample is too small for the difference to be statistically confirmed.

## Retrieval-Augmented Generation

Retrieval-augmented generation (RAG) and web search capabilities in modern LLMs could in principle erode the knowledge asymmetry. Several considerations limit this objection. The timeline is inconsistent with real-time retrieval acting as a corrective during the critical mass-deployment window. ChatGPT launched in November 2022 without web access. The optional Browsing plugin was not available until March 2023 and required explicit user activation, and GitHub Copilot, the dominant code completion tool, does not retrieve external documentation by default. Even where retrieval is technically available, its practical adoption for routine coding tasks is limited. Gao et al. (2024) document persistent limitations in deployed RAG systems. Retrieval quality is unreliable for technical queries, latency costs are substantially higher than parametric inference, and accuracy degrades with longer retrieved contexts. Finally, the empirical trajectory is inconsistent with retrieval acting as a corrective. If real-time retrieval were eroding the Knowledge Wall, the implementation gap should narrow or close as web-enabled models become standard. Figure 5.5 shows the opposite, with the gap widening through early 2026.

# Chapter D – Scope Conditions and Detailed Limitations

## Local Identification

The estimated effect is a Local Average Treatment Effect at the September 2021 cutoff boundary. It identifies the expected difference in diffusion for a library released just after the cutoff relative to one released just before. This is not necessarily the effect for libraries released far from the cutoff. The local estimate is the most defensible claim, and extrapolation to the full ecosystem is speculative.

## Single Cutoff

The design exploits one documented cutoff date. Different cutoffs, for Claude, Gemini, Llama, and subsequent GPT versions, would produce overlapping exclusion windows that are harder to isolate. Whether the findings generalize to other cutoff dates is an open empirical question that would require a multi-cutoff design.

## Python Specificity

Python is a particularly LLM-influenced ecosystem. It is the dominant language for data science and machine learning, where AI-assisted coding is most prevalent. The effect size identified here may be larger than what would be found in languages or ecosystems with less AI-mediated developer activity. Replication in other language ecosystems, for example npm for JavaScript or crates.io for Rust, would establish whether the Knowledge Wall is a general phenomenon or one specific to the AI-adjacent Python community.

## Emergence Stage

The analysis focuses on libraries in their first weeks and months of existence. Established libraries with large user bases are less susceptible to the Knowledge Wall because they achieve diffusion through mechanisms independent of LLM recommendation, including direct network effects, conference presentations, and existing dependency chains.

## Multiple Testing

The paper reports RDD estimates across multiple outcomes (52-week downloads, post-AI downloads, several GitHub import horizons), three success tiers, and two designs (main RDD and Diff-in-RDD). No family-wise error correction is applied. The probability of observing at least one significant result by chance across this many tests is non-trivial. The primary defense is that the baseline-adjusted Diff-in-RDD for the Successful tier ($p = 0.002$) is designated as the main specification *ex ante*, the activation pattern is consistent across horizons, and the direction of effects is stable across tiers. Even so, marginal results should be read with extra care.

## Secular Growth

The Diff-in-RDD assumes that the seasonal pattern at the September boundary is stable across 2018–2021. If the Python ecosystem experienced accelerating growth over this period, the 2018–2020 placebo baseline would understate the expected 2021 seasonal jump and bias the excess jump estimate away from zero. The placebo cohorts do not show a monotonic trend in their individual discontinuity estimates, which is some reassurance, but a formal test for trend in placebo discontinuities has not been conducted.

# Chapter E – Bandwidth and Donut Robustness

The main Diff-in-RDD estimates are stable across $h \in \{13, 15, 18, 26\}$ weeks, both in sign and in approximate magnitude. Results at $h = 10$ are too imprecise to be informative, because the donut exclusion compresses the effective estimation sample, and they are not part of the primary evidence base. Figure E.1 plots the excess jump across bandwidths.

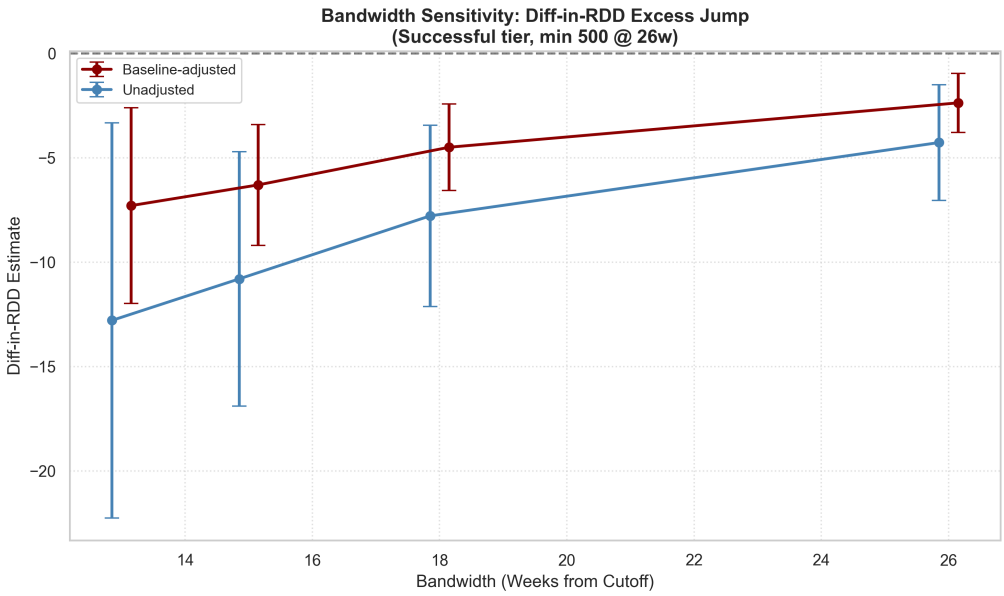


Figure E.1 - Bandwidth sensitivity of the Diff-in-RDD excess jump (Successful tier, min. 500 downloads at 26 weeks). Points are WLS point estimates; vertical bars are 95% confidence intervals from year-by-week cluster-robust standard errors. The estimate is negative and stable across $h \geq 13$. *Source:* Author's calculations based on PyPI data.

The single-year GitHub `rdrobust` estimates are substantially more sensitive to bandwidth than the PyPI estimates. At $h = 13$ weeks the post-AI import discontinuity is 26.8 ($p = 0.014$); at $h = 18$ it falls to 2.9 ($p = 0.129$). This instability motivated the GitHub Diff-in-RDD reported in Table 5.6, which nets out seasonal baselines and yields more stable but smaller estimates (all-time excess jump $= 2.1$, $p = 0.060$). The properly identified GitHub estimates are directionally consistent with the channel interpretation but do not reach conventional significance, reflecting the limited size of the matched subsample.

Replacing the asymmetric 9-week donut with a symmetric $\pm 4$-week exclusion gives qualitatively similar results. The post-AI download discontinuity for the Broad tier remains negative and highly significant (Estimate $= -4.35$, $p < 0.001$), so the Knowledge Wall does not appear to be an artifact of the particular exclusion window.

# Chapter F – Mechanism Moderation Test

A direct mechanism test asks whether suppression is heterogeneous in AI exposure intensity. The specification is

$$Y_i = \alpha + \tau D_i + \gamma \cdot \text{HighAI}_i + \delta \cdot (D_i \times \text{HighAI}_i) + f(X_i) + \varepsilon_i,$$

where $\text{HighAI}_i$ indicates libraries above the median of the commit-level AI score (`avg_ai_score_52wk`) within the GitHub-matched subsample. The coefficient of interest is $\delta$, the differential suppression for libraries that show up more often in AI-generated code.

Table F.1 - AI Exposure Moderation (Exploratory)

| **Outcome** | **Adjusted** | **Interaction ($\delta$)** | **SE** | **p-value** | **N** |
|---|---|---|---|---|---|
| 52-week Downloads | No | 0.396 | 0.295 | 0.179 | 1,303 |
| 52-week Imports | No | 0.257 | 0.292 | 0.380 | 1,303 |
| 52-week Imports | Yes | 0.165 | 0.273 | 0.546 | 1,303 |
| Post-AI Downloads | No | 0.245 | 0.598 | 0.683 | 1,303 |
| Post-AI Downloads | Yes | −0.312 | 0.469 | 0.505 | 1,303 |
| Post-AI Imports | No | 0.011 | 0.513 | 0.984 | 1,303 |
| Post-AI Imports | Yes | −0.110 | 0.490 | 0.822 | 1,303 |

*Notes:* Interacted WLS with week-clustered standard errors. Triangular kernel, $h = 26$ weeks, no donut exclusion (the full bandwidth window is used because the exploratory subsample is already small; $N = 1{,}303$). "Adjusted" specifications control for $\log(1 + \text{52-week downloads})$; omitted for the 52-week downloads outcome (circularity). All estimates are on the $\log(1 + Y)$ scale.
*Source:* Author's calculations based on PyPI and GitHub data.

None of the interactions reach conventional significance. With $N = 1{,}303$ and roughly 650 observations per AI-exposure bin, the interacted split-sample RDD is almost certainly underpowered to detect a moderation effect of the size we would expect. The right way to read these nulls is as a power problem, not as positive evidence that exposure does not matter. The data simply cannot tell apart a broad ecosystem-wide penalty from a sharper effect on AI-mediated usage that this test does not have the resolution to find.

There is also an endogeneity caveat. The moderator `avg_ai_score_52wk` is measured after treatment and is plausibly affected by it. A library suppressed by the Knowledge Wall will attract fewer imports overall, and in particular fewer AI-generated imports, simply because LLMs know less about it. So the moderator partly absorbs the treatment, which limits how much causal weight the interaction can carry.

# Chapter G – Raw Long-Horizon Trajectory

Figure G.1 plots the raw (unnormalized) weekly average PyPI downloads for the July 2021 (pre-cutoff) and October 2021 (post-cutoff) cohorts. It is the level-counterpart to the normalized ratio plot in the main text (Figure 5.5).

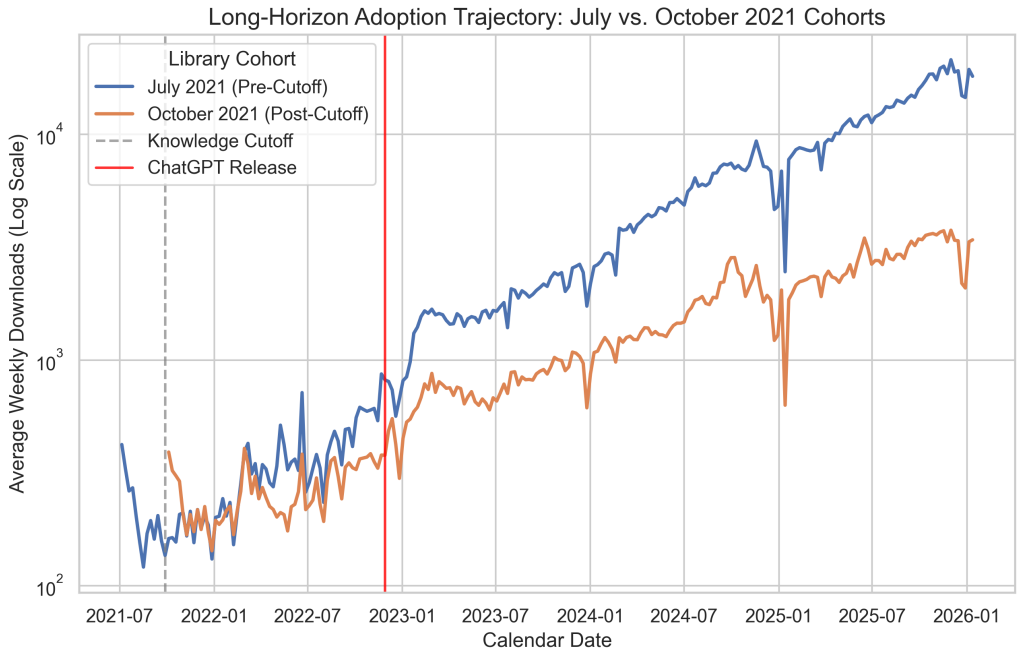


Figure G.1 - Weekly average PyPI downloads for the July 2021 (pre-cutoff) and October 2021 (post-cutoff) cohorts, from release through January 2026 (log scale). Dashed vertical lines mark the release of ChatGPT (November 2022) and GPT-4 (March 2023). The gap between cohorts opens after ChatGPT and persists through the end of the observation window, with no sign of catch-up. *Source:* Author's calculations based on PyPI data.